

WW Lab

A self-paced course on WW3, NOAA's third-generation spectral wave model, from the wave spectrum to GPUs, WW4 and SWAN

Matheus Hoffmann

2026-09

Contents

1	00. Orientation: what WW3 actually computes	1
1.1	The one equation	1
1.2	The source terms	2
1.3	Phase-averaged, not phase-resolving	2
1.4	The five dimensions	2
1.5	The program pipeline	3
1.6	Two input file formats	3
1.7	Where to go next	3
2	01. Getting it built	4
2.1	Two halves	4
2.2	Dependencies	4
2.3	Building inside the pinned toolchain	5
2.4	Switch files are compile options	5
2.5	Build by hand	6
2.6	Prove it works	7
2.7	Things that go wrong	7
2.8	Building with nvfortran	7
3	02. Anatomy of a run	8
3.1	ww3_grid: the most important program	8
3.2	ww3_strt: initial conditions	9
3.3	ww3_prnc: forcing	10
3.4	ww3_shel: the model	10
3.5	ww3_ounf / ww3_ounp: output	10
4	03. Grids, bathymetry, masks	12
4.1	Four grid types	12
4.2	Grid closure	12

4.3	Bathymetry	12
4.4	Generating the files	13
4.5	Masks	13
4.6	Obstruction grids: the underrated one	14
4.7	Choosing resolution	14
4.8	Exercise	14
5	04. Forcing: winds, currents, ice, water levels	15
5.1	What WW3 will accept	15
5.2	Winds are everything	15
5.3	Getting the winds: GFS via <code>get_gfs.sh</code> and <code>ecCodes</code>	16
5.4	<code>ww3_prnc</code>	16
5.4.1	The four things that go wrong	17
5.4.2	Verify before you run the model	17
5.5	Boundary spectra	17
5.6	Exercise	18
6	05. Nesting and multi-grid	19
6.1	One-way nesting: chained <code>ww3_shel</code> runs	19
6.2	Two-way: <code>ww3_multi</code>	20
6.3	Which to use	20
6.4	Unstructured as an alternative	20
7	06. Output and post-processing	21
7.1	The seven output types	21
7.2	Fields worth knowing	21
7.3	Partitioning: the thing to actually understand	22
7.4	Reading the output	22
7.4.1	<code>ww_fetch_analyse</code> : the example-01 payoff	23
7.4.2	Compare two runs with <code>nccmp-tol</code>	23
7.5	Validation	24
8	07. Physics choices: which source terms, and why it matters	25
8.1	The source-term packages	25
8.2	Why this matters more than you'd like	25
8.3	The nonlinear term	26
8.4	Propagation	26
8.5	Shallow water	26

8.6	Sea ice	27
8.7	Tuning namelists	27
8.8	A note on tuning	27
8.9	Why W3SNL1 is the first kernel to port	28
9	08. The Python ecosystem, and why this repo does not depend on it	29
9.1	The landscape	29
9.2	What pyww3 gets right	29
9.3	Why this repo does not depend on any of it	30
9.4	What we use instead	30
10	09. Measure first: benchmark, profile, compile options, run configuration	31
10.1	The reference run	31
10.2	The metric	31
10.3	bench/	32
10.4	Profile	32
10.5	WW3's own matrix: the bit-for-bit gate	33
10.6	Step 1: the compile-option matrix	33
10.7	MPI × OpenMP on one node	33
10.8	Step 2: run-configuration knobs, one at a time	34
10.9	When reordering breaks b4b: nccmp-tol	34
10.10	Sources	34
11	10. Modern Fortran refactoring: the step before any port	35
11.1	What a profile-top routine looks like	35
11.2	The checklist	36
11.3	DO CONCURRENT	36
11.4	Killing automatic arrays	36
11.5	The runtime switch: one binary, two paths	37
11.6	The per-routine parity test	37
11.7	The contrast: directives	38
11.8	Checklist for one routine	38
12	11. Kokkos and modern C++ for Fortran people	39
12.1	Execution and memory spaces	39
12.2	View	40
12.3	Layout: LayoutLeft is Fortran order	40

12.4Mirrors and deep_copy	40
12.5parallel_for, parallel_reduce, parallel_scan	40
12.6TeamPolicy and scratch	41
12.7KOKKOS_LAMBDA and what a kernel may touch	41
12.8Fences	41
12.9float versus double	41
12.10Modern C++ hygiene, briefly	42
12.11bind(C) interop	42
12.12Building: presets and the CUDA backend	42
1312. Porting a kernel: W3SNL1	44
13.1Source map	44
13.2Config and Tables	44
13.3Index conventions	44
13.4Kernel structure	45
13.5The shim and W3KOKKOSMD	45
13.6Fixtures	46
13.7L1 tests and their tolerances	46
13.8The optional ww3_lib cross-check	47
13.9PATCH.md: what changes in the WW3 caller	47
13.102: replay a regtest through both paths	47
13.11The timing line	48
13.12What "done" means	48
1413. Bulk porting with agents	49
14.1The FESOM2 recipe	49
14.2What a port PR contains	50
14.3The ranked list and the phases	50
14.4Interoperability, one binary	50
14.5The data-residency ladder	51
14.6The ladder and its gates	51
14.7The operation decision	52
14.8Prompting an agent: patterns and anti-patterns	52
14.9WW4: do not compete, build in its shape	53
14.10Sources	53

15 14. WW4: what's coming, and how mature it is	54
15.1 It exists	54
15.2 Maturity: a snapshot	54
15.3 Why a new model rather than WW3 v8	55
15.4 Languages: the contentious part	56
15.5 Yes, it is being written partly by AI agents	56
15.6 Timeline	57
15.7 What WW4's test levels mean for a WW3 port	57
15.8 What this means for WW3: and for you	57
15.9 One thing that must survive	58
16 15. SWAN: the other one you should know	59
16.1 What it is	59
16.2 Where the source actually lives	59
16.3 Building it	60
16.3.1 Switches, SWAN-style	60
16.4 SWAN versus WW3: the actual difference	60
16.5 When to use which	61
16.6 The ecosystem around it	61
16.7 Exercise	61

Chapter 1

00. Orientation: what WW3 actually computes

1.1 The one equation

WW3 solves the **spectral action density balance equation**. At every point in space and every moment in time, it carries a two-dimensional spectrum: how much wave energy sits at each frequency and each direction.

$$\frac{\partial N}{\partial t} + \nabla_{\mathbf{x}} \cdot (\dot{\mathbf{x}} N) = \frac{S}{\sigma}$$

- $\frac{\partial}{\partial k} (\dot{k} N) + \frac{\partial}{\partial \theta} (\dot{\theta} N) = \frac{S}{\sigma}$

where $N(k, \theta; \mathbf{x}, t) = F/\sigma$ is **action** density (energy over intrinsic frequency, conserved in the presence of currents, where energy isn't).

Read the terms left to right and you have the whole model:

Term	Physics	WW3 switch/flag
$\partial N / \partial t$	evolution in time	—
$\nabla_x \cdot (\dot{x} N)$	waves travel at the group velocity	RUN%FLCX, RUN%FLCY
$\partial(\dot{k} N) / \partial k$	shoaling, current-induced frequency shift	RUN%FLCK
$\partial(\dot{\theta} N) / \partial \theta$	refraction by depth and current	RUN%FLCTH
S / σ	all the physics you can't derive	RUN%FLSOU

The left side is clean conservative transport in a 4D phase space (x, y, k, θ). It's numerics, and it's solvable. The right side is where the model earns its keep and where all the uncertainty lives.

The first half of this course (00–08) teaches you to *run* the thing that solves this equation. The second half (09–13) teaches you to make it run *faster without changing the answer*, and it follows the ladder of the UFRJ/DEL project proposal that this repository serves: compile options → run configuration → modern Fortran → C++/Kokkos kernels for the GPU, each rung gated by parity against the reference run before the next one is allowed to start. The proposal itself is in [../pubs/proposal/pt/](#) (Portuguese, eight short sections) and its mind maps in [../pubs/proposal/mapas-mentais.pt.md](#);

the lessons are the English, hands-on version of the same ladder, with the code in [../kokkos/](#).

1.2 The source terms

$$S = S_{in} + S_{nl} + S_{ds} + S_{bot} + S_{db} + S_{ice} + \dots$$

- S_{in} : **wind input**. Momentum from the atmosphere into the waves. The parameterisation you pick (ST2, ST4, ST6) mostly means picking a version of this.
- S_{nl} : **nonlinear wave-wave interaction**. Four-wave resonant quartets shuffle energy between frequencies. This is the term that makes the spectrum evolve towards its characteristic shape rather than just growing where the wind pushes. The exact computation is $\sim 1000\times$ the cost of everything else combined, so everybody uses the Discrete Interaction Approximation (NL1, Hasselmann 1985) and everybody knows it's the weakest link.
- S_{ds} : **dissipation**, mostly whitecapping. Historically the tuning term: whatever was needed to make $S_{in} + S_{nl}$ balance out to observed spectra.
- S_{bot} , S_{db} : bottom friction, depth-induced breaking. Shallow water only.
- S_{ice} : ice attenuation and scattering (IC1-IC5, IS1/IS2).

The thing to internalise: S_{in} and S_{ds} are individually large and individually uncertain, and they nearly cancel. The net is a small difference of big numbers. That's why source-term packages come as matched sets: you cannot mix the ST4 input with the ST6 dissipation and expect anything sensible.

1.3 Phase-averaged, not phase-resolving

WW3 does **not** know where any individual wave crest is. It tracks statistics. If you want a surface you can render, or wave-by-wave runup on a beach, or slamming loads on a hull, you want a different class of model entirely (SWASH, XBeach, FUNWAVE, OpenFOAM; see docs/AWESOME-WW3_202609.md).

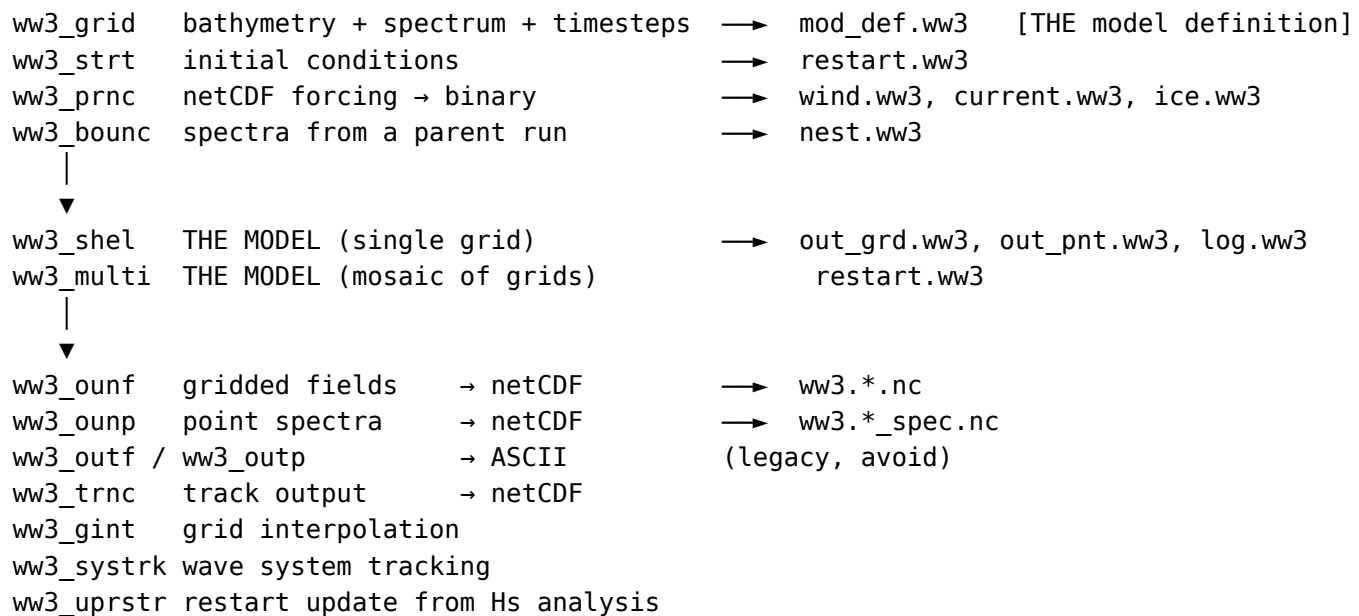
What WW3 gives you is: the spectrum, and everything derivable from it. H_s , peak period, mean direction, directional spread, Stokes drift, radiation stresses, partitioned swell systems.

1.4 The five dimensions

Every field WW3 carries lives on **latitude $\times$ longitude $\times$ frequency $\times$ direction $\times$ time**. A modest regional run (81×81 points, 32 frequencies, 36 directions) is 7.5 million numbers *per timestep*, before any output. This is why WW3 is expensive, and it is directly relevant to the GPU question in lessons 11–13: you cannot casually shuttle that array across PCIe every step, so a port has to decide where the spectrum *lives*.

1.5 The program pipeline

WW3 isn't one binary. It's a family, communicating through files:



mod_def.ww3 is the centre of gravity. It's an opaque binary produced by `ww3_grid` containing the complete model definition: grid, spectral discretisation, timesteps, physics configuration. Every other program reads it. Change `ww3_grid.nml`, and every downstream artifact is stale until you rerun `ww3_grid`. A large share of all confusing WW3 behaviour reduces to a stale `mod_def`.

1.6 Two input file formats

WW3 accepts **legacy** `.inp` (fixed-format, positional, comment lines start with `$`) and **modern** `.nml` (Fortran namelists, order-independent, self-documenting). Both work in v7. Everything in this repo uses `.nml`, because:

- the annotated templates in `$WW3/model/nml/` document every single parameter inline;
- you can omit anything you want defaulted;
- the small Fortran generators in `examples/` write them, and so does every third-party wrapper you might meet (lesson 08). Nobody targets `.inp` any more.

If you follow an older tutorial and get error reading input file, check whether it's handing an `.inp` to a program expecting `.nml` or vice versa. That's a real and common failure mode.

1.7 Where to go next

→ [01-build.md](#): get it compiled.

Chapter 2

01. Getting it built

2.1 Two halves

The WW3 package is **the git repo plus a binary data bundle**. The repo has the Fortran and the regression-test *configurations*; the bundle has the bathymetry, forcing, and reference output the tests need. The bundle lives on NOAA's FTP, not in git.

```
git clone --branch develop https://github.com/NOAA-EMC/WW3.git ~/src/WW3
cd ~/src/WW3
./model/bin/ww3_from_ftp.sh      # the other half. Slow. Required for most regtests.
export WW3=$PWD
```

just get does the clone for you (into ~/src/WW3; WW3_DATA=1 just get also fetches the bundle). The repo also carries a fork of WW3 as the WW3/ submodule, pinned to one revision. That is the tree the Kokkos port in lessons 12–13 is written against.

Don't use the GitHub release tarball. The newest tagged release is 6.07.1, from April 2019. Real work happens on develop and main, and the wider community is running v7.14.x. Clone a branch.

2.2 Dependencies

- Fortran 90 compiler: gfortran, ifort/ixf, nvfortran, or Cray
- **CMake ≥ 3.19**
- **NetCDF ≥ 4.1.1 with Fortran bindings.** The C library alone won't do; ww3_ounf and friends need netcdf.mod. On Debian/Ubuntu: libnetcdf-dev.
- MPI for anything realistic

bash scripts/00_prereqs.sh (just prereqs) installs all of it on Debian/Ubuntu. The recommended route, though, is the next section: every number in this course was produced inside one pinned toolchain, and "it works on my machine" is exactly the disease a benchmarking project cannot afford.

2.3 Building inside the pinned toolchain

The compilers and libraries come from one locked nixpkgs revision, via the nix-config/labs/prati flake (a sparse git submodule; just submodule-init after a fresh clone). Three recipes cover almost everything:

```
just ww3          # enter the toolchain-only shell  (== nix develop ./nix-config/labs/
just toolchain    # print the exact pinned versions
just build [switch] # full rebuild of $WW3 with a switch file (default: switches/switch_
```

What the shell pins (v) (this is just toolchain on the lab machine; see the root README.md):

Component	Pinned
gfortran	GNU Fortran (GCC) 15.3.0
MPI	Open MPI 5.0.10
NetCDF	netcdf-c 4.10.1, netcdf-fortran 4.4.6-development, HDF5 1.14.6
METIS / ParMETIS	5.2.1 / 4.0.3 (for PDLIB unstructured runs)
ecCodes	2.48.0 (GRIB → netCDF, lesson 04)
CMake	4.4.2
Kokkos / GoogleTest	5.2.0 / 1.18.0 (lessons 11–12; see kokkos/README.md)

just build is scripts/02_build_ww3.sh run inside that shell: it deletes build/, configures with -DSWITCH=<file> -DCMAKE_BUILD_TYPE=Release, and runs make -j. It always starts from scratch, on purpose (see the next section for why). A third argument selects Debug. just rt builds with a regtest's own switch file and runs the test (~30 s for ww3_tp1.1), which is the fastest “is the toolchain sane” check there is.

The same shell carries Kokkos and GoogleTest, so just kokkos-test serial-debug (lesson 11) builds against exactly the compilers WW3 was built with. That is not a convenience; when lesson 12 claims the C++ kernel reproduces the Fortran *bit for bit*, the claim is only meaningful because both sides came out of the same pinned gfortran and the same flags.

2.4 Switch files are compile options

Before you build you pick a **switch file**: a whitespace-separated list of CPP keys that selects the physics and the parallelism at compile time. See [../switches/README.md](#) for the full group-by-group breakdown. Here is switches/switch_lab_shrd, the lab default, one key at a time:

Key	Group	What you are choosing
F90	language	Fortran 90 style and system calls. Always.
NOGRB	GRIB output	None. The alternatives need NCEP's GRIB libraries.
NOPA	coupling	No external coupler (NUOPC/OASIS off).
LRB4	record length	4-byte record-length units for the binary files (mod_def.w
NC4	netCDF	Legacy “enable netCDF-4 output” key. Inert in 7.14 (v
SHRD	parallelism	Shared memory, i.e. a serial binary. The alternative is D

Key	Group	What you are choosing
PR3 UQ	propagation	Third-order ULTIMATE QUICKEST with the Garden Spr
FLX0	air-sea flux	No separate flux routine: ST4 computes its own stress in
LN1	linear input	Cavaleri & Malanotte-Rizzoli seeding, so a spectrum can
ST4	source terms	Ardhuin et al. 2010 input and dissipation. Lesson 07.
STAB0	stability	No air-sea stability correction on the wind input.
NL1	nonlinear	The Discrete Interaction Approximation: W3SNL1, the first
BT1	bottom friction	JONSWAP empirical.
DB1	depth breaking	Battjes-Janssen.
MLIM	limiter	Miche-style shallow-water Hs limiter.
TR0 BS0	triads, bottom scattering	Off.
IC0 IS0	sea ice	No ice dissipation, no ice scattering.
REF0	reflection	No shoreline reflection.
XX0	extra source term	No user-defined/experimental term compiled in.
WNT1 WNX1	wind interpolation	Linear in time, linear in space.
RWND	wind	Wind is taken relative to the current.
CRT1 CRX1	current interpolation	Linear in time, linear in space.
00-07, 014	output/log verbosity	Extra diagnostics in log.wv3 and stdout. Harmless; 07 gi

Two things to internalise:

1. **Every key is a compile-time decision.** The build system preprocesses the .F90 sources against these keys, so **changing the switch file requires a full rebuild**: not make, a full rm -rf build (which is what just build does). "I turned on ST4 and nothing changed" is nearly always a stale build directory.
2. **That makes the switch file the first rung of the optimisation ladder.** Together with the compiler flags it is a *matrix* (SHRD vs DIST MPI vs OMPG OMPH, -O2 vs -O3 -march=native), and each cell of that matrix either reproduces the reference run bit for bit or it doesn't. Lesson 09 builds that matrix and shows how WW3's own regression suite is used as the gate.

2.5 Build by hand

If you would rather not use just:

```
cd $WW3
rm -rf build && mkdir build && cd build
cmake .. -DSWITCH=/abs/path/to/ww3-gpu/switches/switch_lab_shrd
make -j$(nproc)
ls bin/
```

-DSWITCH takes either a bare name resolved inside \$WW3/model/bin/, or an absolute path to your own file.

Useful variants:

```
cmake --build . --target ww3_shel # rebuild one program
cmake .. -DCMAKE_BUILD_TYPE=Debug -DSWITCH=... # bounds checking, backtraces
export NetCDF_ROOT=$(nf-config --prefix) # if CMake can't find NetCDF
```

2.6 Prove it works

```
just retest ww3_tp2.2
```

```
# or: bash scripts/03_run_retest.sh $WW3 ww3_tp2.2
```

That script runs each program in turn and tees its stdout, rather than hiding everything behind the `run_test` harness. Watch what each one prints. `ww3_grid` in particular emits a long, genuinely useful summary of the grid it built and every namelist it read. Always keep it: `ww3_grid | tee ww3_grid.out`.

2.7 Things that go wrong

Symptom	Cause
Cannot open include file 'netcdf.inc' / undefined netCDF symbols	NetCDF Fortran bindings
ww3_ounf/ww3_ounp are not built at all	CMake did not find netCDF
ww3_grid demands an obstruction file you don't have	FLAGTR in namelists.nml is
Model runs, output is all zeros	Forcing never arrived. C
Changed physics, nothing changed	Stale build/. rm -rf buil
Changed the grid, downstream programs behave oddly	Stale mod_def.ww3. Rerun
error reading input file	.inp vs .nml mismatch.
nix develop complains the submodule is missing	just submodule-init.

2.8 Building with nvfortran

Possible and worth doing as a CPU build, but you must rebuild HDF5 and NetCDF with `nvfortran` first: distro packages are `gfortran`-built and their `.mod` files are unusable. [../gpu/build_netcdf_nvfortran.sh](#) sketches it and [../gpu/README.md](#) explains the sandbox around it. Lesson 10 uses the `gpu/` directive examples as the contrast that motivates Kokkos; the GPU port itself (lessons 11–13) does not need `nvfortran` at all: it is C++ compiled with `nvcc` inside the pinned `#cuda` shell.

→ [02-anatomy-of-a-run.md](#)

Chapter 3

02. Anatomy of a run

Work through [examples/01-fetch-limited-growth](#) with this open. Every claim here is visible in those files.

3.1 ww3_grid: the most important program

It reads `ww3_grid.nml` plus your ASCII bathymetry/mask, and writes `mod_def.ww3`. Its namelist blocks, in dependency order:

&SPECTRUM_NML: the spectral discretisation.

```
SPECTRUM%FREQ1 = 0.04118    ! lowest frequency, Hz
SPECTRUM%XFR    = 1.1        ! geometric ratio between bins
SPECTRUM%NK     = 32         ! number of frequencies
SPECTRUM%NTH    = 24         ! number of directions
```

$f_{\text{max}} = \text{FREQ1} \times \text{XFR}^{(\text{NK}-1)}$. With the defaults that's $0.04118 \rightarrow 0.76$ Hz, i.e. periods from 24 s down to 1.3 s. **Energy outside this range does not exist in your model.** If you care about 26-second Southern Ocean swell, lower FREQ1. Nothing warns you.

NTH = 24 means 15° bins. That's coarse for swell. Coarse direction resolution produces the **Garden Sprinkler Effect**: a smooth swell field propagating far from its source breaks up into discrete beams, because each direction bin travels as an independent ray. The PR3 propagation scheme includes a correction; more directions helps more.

&RUN_NML: switch individual terms of the balance equation on and off. Turning `FLSOU = F` gives pure propagation, which is how the `ww3_tp*` regression tests isolate numerics from physics. Extremely useful for debugging.

&TIMESTEPS_NML: four numbers, and they are not tuning knobs.

```
DTMAX    global timestep: everything else divides into it
DTXY     spatial propagation, CFL-limited
DTKTH    refraction / wavenumber shift
DTMIN    minimum source-term integration step
```

The CFL limit comes from the fastest group velocity, which belongs to the *lowest* frequency (deep water: $c_g = g/(4\pi f)$, so low $f \rightarrow$ fast):

```
cg_max = g / (4π·FREQ1)
T_cfl  = Δx_min / cg_max
DTXY   ≈ 0.9 · T_cfl      (round DOWN)
DTMAX  ≈ 3 · DTXY
DTKTH  ≈ DTMAX / 2       (÷10 if you have strong currents)
DTMIN  ≈ 10 s
```

Keep them integer multiples of each other. Round down, never up.

&GRID_NML: TYPE is RECT / CURV / UNST; COORD is SPHE / CART; CLOS is NONE / SMPL (periodic in i) / TRPL (tripole, for global grids that need to handle the North Pole). ZLIM is the depth above which a point is permanently land; DMIN is the floor applied to depth in the physics so shallow points don't blow up.

&DEPTH_NML / &MASK_NML / &OBST_NML: ASCII input arrays. Three fields to get right:

- SF: scale factor, **multiplied** into the values you supply. **Depths must be negative below mean sea level.** If your file has positive depths, set SF = -1.
- IDLA: layout. 1 = line by line from the *bottom* row; 3 = line by line from the *top*. Getting this wrong flips your continent upside down and is not always obvious.
- IDFM / FORMAT: free vs fixed format.

Mask legend: 0 land, 1 sea, 2 **active open boundary**, 3 excluded, 7 ice, -1/-2 ice-excluded.

&INBND_COUNT_NML / &INBND_POINT_NML: where boundary spectra enter. Two rules: boundary points must be strictly *inside* the grid (never the first or last row/column, the propagation stencil needs a cell outside them), and CONNECT = T auto-fills every point on the straight line from the previous entry.

GRID%NML = 'namelists.nml': points at a second file holding the physics tuning namelists (&MISC, &SIN4, &SDS4, &PR03, &UNST, ...). Blocks not needed by your switch settings are skipped automatically. If you have no obstruction grid, &MISC FLAGTR = 0.

Run it, always keeping stdout:

```
ww3_grid | tee ww3_grid.out
```

It prints the whole grid summary, every namelist value it read, and the count of sea/land/ boundary points. That count is your first sanity check.

Outputs: mod_def.ww3 (binary, the model definition), mapsta.ww3 (status map), mask.ww3 (ASCII land/sea; open it and look at it).

3.2 ww3_strt: initial conditions

Writes restart.ww3. If it's absent, ww3_shel cold-starts from a calm sea and says so in the log, which is usually what you want for a spin-up run. For a specific initial

spectrum (the propagation tests seed a Gaussian packet and watch it travel), copy an `ww3_strt` input from a regtest: `$WW3/regtests/ww3_tp1.1/input/` is the canonical one.

3.3 `ww3_prnc`: forcing

netCDF → WW3 binary, interpolated onto your grid. One invocation per forcing type; you swap `ww3_prnc.nml` between runs. The failure modes are boring and universal: wrong variable names, wrong coordinate names, wrong longitude convention (0-360 vs -180-180), wrong time units. `ncdump -h` before you start.

3.4 `ww3_shel`: the model

Reads `ww3_shel.nml` and `mod_def.ww3`.

- **&DOMAIN_NML**: start and stop, 'YYYYMMDD HHMMSS'.
- **&INPUT_NML**: per forcing: 'T' read from file, 'H' homogeneous (given below), 'C' from a coupler. Boundary spectra are *not* flagged here: `nest.ww3` is picked up automatically if present.
- **&OUTPUT_TYPE_NML**: what to output. Seven output types exist: gridded fields, point spectra, track output, restarts, boundary data, separated wave fields, coupling fields.
- **&OUTPUT_DATE_NML**: when. Each entry is START STRIDE_SECONDS STOP. A stride of '0' disables that output type.
- **&HOMOG_COUNT_NML** / **&HOMOG_INPUT_NML**: constant forcing without any data files. For 'WND': speed, direction, air-sea ΔT .

Watch `log.ww3` while it runs. It prints a timestep table with a column per input showing which forcings updated when: X for an update, F for a file read. If a column never moves, your forcing isn't arriving.

For MPI builds: `mpirun -np N ww3_shel`.

3.5 `ww3_ounf` / `ww3_ounp`: output

`ww3_ounf` turns `out_grd.ww3` into gridded netCDF. `ww3_ounp` turns `out_pnt.ww3` into spectral netCDF.

FIELD%TIMESPLIT controls file chunking: 0 one file, 4 yearly, 6 monthly, 8 daily, 10 hourly. FIELD%TYPE is 2 (packed SHORT), 3 (mixed), 4 (REAL). Use 4 while learning so you never wonder whether an odd value is a packing artefact.

The field name list is long and documented at the top of `$WW3/model/nml/ww3_ounf.nml`. The ones you'll use constantly:

HS	significant wave height	T01/T02/T0M1	mean periods
FP	peak frequency	DIR SPR	mean direction / spread

WND	wind (echo it back, always)	DPT	depth
PHS	PTP PDIR PSPPR	partitioned wind sea + swell	systems
EF	1D frequency spectrum	USS TUS	Stokes drift
SXY	radiation stresses	UST	friction velocity

Always output WND and DPT. They cost nothing and they catch forcing and grid errors immediately.

→ [03-grids.md](#)

Chapter 4

03. Grids, bathymetry, masks

4.1 Four grid types

Type	GRID%TYPE	When
Rectilinear	RECT	Regular lat-lon or Cartesian. Default. Cheapest per point.
Curvilinear	CURV	Body-fitted logically-rectangular mesh; you supply x and y coordi
Unstructured	UNST	Triangular mesh from GMSH .msh. Resolution where you need it,
SMC	via &SMC_NML	Spherical Multiple-Cell: quad-tree refinement on a sphere. Used

Unstructured is where coastal work is going, because the resolution mismatch between a 1000 km shelf and a 200 m inlet is brutal on a rectilinear grid. The cost: unstructured WW3 uses implicit schemes and needs PDLIB + ParMetis for domain decomposition (added v6.04), which is a build-time complication.

4.2 Grid closure

GRID%CLOS matters only for global spherical grids:

- NONE: no wrap. Regional.
- SMPL: periodic in i ; $(NX+1, J) \rightarrow (1, J)$. Forces $SX = 360/NX$.
- TRPL: tripole. Periodic in i and folded at $j = NY+1$: $(I, NY+1) \rightarrow (NX-I+1, NY)$. Requires NX even and a curvilinear grid. This is how you avoid the North Pole singularity on a global grid.

4.3 Bathymetry

Sign convention: negative below mean sea level. This is opposite to how most people think about "depth" and it is the single most common grid error. GEBCO's elevation is already negative below sea level, so $DEPTH\%SF = 1$. For a file of positive depths, $DEPTH\%SF = -1$.

IDLA decides which way is up. 1 = bottom row first, 3 = top row first. Check by comparing `mask.wv3` (ASCII, written by `ww3_grid`) against a map. Do it visually, once, every time you build a new grid.

ZLIM vs DMIN. ZLIM (e.g. -0.10) is the coastline: points shallower than this are permanently excluded and will never be wet even if water level rises. DMIN (e.g. 2.50) is a floor applied to the depth used in the physics, so a nearly-dry cell doesn't produce division-by-zero. They do different things and both matter.

4.4 Generating the files

`ww3_grid` does not read netCDF bathymetry. It reads plain ASCII arrays, one row per line in the order IDLA declares, and you are expected to produce them. The two examples each ship a small Fortran program for that, and the point of generating the files rather than committing them is that you can change NX, NY or the depth and rerun, which is exactly what the exercises ask you to do.

Example	Program	Writes	How
01-fetch-limited-growth	<code>make_inputs.F90</code> , no arguments	<code>depth.inp</code> , <code>mask.inp</code>	A 61×5
02-regional-real-forcing	<code>make_bathy.F90</code> , <code>make_bathy gebco.nc</code>	<code>bathy.inp</code> , <code>mask.inp</code>	Reads a

Each example's `run.sh` compiles its generator with `gfortran` if the binary is missing, runs it, and then runs the WW3 programs. Read the sources: they are short, and the mask legend and the IDLA convention are spelled out in the comments.

Nearest-neighbour sampling is a deliberate downgrade from interpolation. Point-sampling a 15-arc-second dataset onto a 0.1° grid throws away most of the information and can miss a whole shoal; for production work you want area-weighted (conservative) regridding, which is a job for a proper regridding tool, not a forty-line program. For a course grid it is fine, and it has one virtue interpolation lacks: every depth in `bathy.inp` is a number GEBCO actually reported.

4.5 Masks

- 2 excluded boundary point (ice-covered)
- 1 excluded sea point (ice-covered)
- 0 excluded land point
- 1 sea point
- 2 active boundary point
- 3 excluded grid point
- 7 ice point

Note 2: boundary points are declared in the `mask`, and/or via `&INBND_POINT_NML` in `ww3_grid.nml`. `ww3_grid` will promote active points on the declared boundary segments to status 2 and report how many.

4.6 Obstruction grids: the underrated one

At 0.1° you cannot resolve a 3 km island. But that island blocks waves. WW3's answer is **subgrid obstruction**: a per-cell transparency in x and y, between 0 (fully blocking) and 1 (fully transparent), applied during propagation.

Controlled by &MISC FLAGTR in namelists.nml:

```
0 no obstruction (and then you must NOT supply an obstruction file)
1 transparency at cell boundaries
2 transparency at cell centres
3 as 1, with continuous ice
4 as 2, with continuous ice
```

Generate these with [gridgen](#) (MATLAB). Without them, archipelagos are invisible and your lee-side wave heights are badly wrong. With them, a coarse global grid produces surprisingly decent results in the Indonesian seas and the Aegean.

4.7 Choosing resolution

Two competing constraints:

1. **Physics.** Resolve the bathymetric features that refract the waves you care about.
2. **CFL.** $\Delta t \propto \Delta x$. Halve the grid spacing and you double the timestep count *and* quadruple the point count: $8\times$ the cost for a 2D grid.

The standard resolution is a nested mosaic: a coarse global grid (0.5°) feeding a regional grid (0.1°) feeding a coastal grid (0.01°). Either as separate runs chained through `ww3_bounc`, or as a single `ww3_multi` run with two-way interaction. See lesson 05.

4.8 Exercise

Build the same domain three ways (0.2° , 0.1° , 0.05°) with correctly recomputed timesteps (edit NX, NY, SX, SY in `make_bathy.F90` and `ww3_grid.nml` together), and run example 02's forcing through all three. Pull Hs at the shelf-break point out of each `ww3.nc` with `ncks -v hs -d longitude,<i>-</i> -d latitude,<j>-</j>` (or `ncdump -v hs` and your eyes) and tabulate the three time series side by side. Where does it converge? Where doesn't it? That answer is specific to your coastline and it's the only honest way to pick a resolution. Lesson 06 shows the tools, and `nccmp-tol` is the wrong tool for this one: the grids differ, so the fields aren't comparable point by point.

→ [04-forcing.md](#)

Chapter 5

04. Forcing: winds, currents, ice, water levels

5.1 What WW3 will accept

Forcing	Why it matters
10 m winds	The engine. Everything else is a correction.
Currents	Refraction, frequency shift, and the Agulhas/Gulf Stream giant-wave
Water levels	Tides. Matters in shallow water, irrelevant offshore.
Ice concentration	Attenuates and scatters.
Ice thickness/floe size	Needed by the more sophisticated ice physics (IC2-IC5).
Mud	Dissipation over muddy bottoms. Niche.
Air density, momentum	Mostly for coupled configurations.

Each takes 'T' (from a file), 'H' (homogeneous constant, given in &HOMOG_INPUT_NML), or 'C' (from a coupler).

5.2 Winds are everything

Wave model error is dominated by wind error, not by wave physics. A 10% wind speed error becomes roughly a 20% error in H_s in a growing sea (energy goes roughly as U^2 in the fetch-limited laws, and worse than that in the growth phase). Before you spend a week tuning BETAMAX, check your winds against scatterometer data.

Practical consequences:

- **Resolution.** GFS or ERA5 at 0.25° cannot represent a squall line or a tight tropical cyclone core. For storm cases you want a higher-resolution atmospheric model, not a finer wave grid.
- **Temporal frequency.** Hourly is the minimum for anything with a front in it. 6-hourly winds smear out exactly the events you care about; the 3-hourly GFS output used below is a compromise.

- **Height.** WW3 wants 10 m winds. If your source gives a different level, convert with a log profile, or better, use `&SIN4 ZWND` / the equivalent for your source-term package to tell WW3 what height it's being given.
- **Stability.** `HOMOG_INPUT(i)%VALUE3` and the `STAB` switch handle air-sea temperature difference. A cold-air outbreak over warm water generates markedly more wave energy than the same wind speed in stable conditions.

5.3 Getting the winds: GFS via `get_gfs.sh` and `ecCodes`

Example 02 takes its winds from NOAA's operational **GFS** at 0.25°, because it is free, needs no account, and the whole pipeline is shell plus `ecCodes`, both of which are in the pinned toolchain (`eccodes 2.48.0 (v)`, lesson 01). One sentence on the alternative: ERA5 is the better *hindcast* product, but the only supported way to fetch it is the Copernicus CDS Python client with a registered API key, and this repo's lab code does not depend on Python (lesson 08), so ERA5 is not used here.

```
cd examples/02-regional-real-forcing
./get_gfs.sh 20240701 00          # cycle date and hour: YYYYMMDD HH
ncdump -h gfs_winds.nc | head -40
```

What the script does, step by step (read it, it is short):

1. Asks NOMADS' `filter_gfs_0p25.pl` for the subset `leftlon=-52 rightlon=-44` `toplat=-24 bottomlat=-32` with `var_UGRD=on var_VGRD=on lev_10_m_above_ground=on`, so you download two fields over one box instead of a 500 MB global GRIB.
2. Repeats that for forecast hours 0–192 every 3 h of the chosen cycle.
3. Concatenates the GRIB messages along time with `cdo mergetime` (`cdo 2.5.1` is in the pinned toolchain (`v`)) and converts with `grib_to_netcdf` → `gfs_winds.nc`.

Two things to notice about what you just got. First, it is a **forecast**, not an analysis: 192 h issued from one cycle. That is fine for learning the pipeline and for “what did the model think would happen”; for a hindcast you stitch the `f000` analyses of successive cycles instead, which is a five-line change to the script. Second, the variable names changed on the way through: GRIB calls them `10u/10v`, and `grib_to_netcdf` writes them as `u10/v10` △: verify with `ncdump -h` before trusting the namelist below, and `grib_ls gfs_*.grb2` shows you the GRIB side of the same fields.

5.4 `ww3_prnc`

```
cp ww3_prnc_wind.nml ww3_prnc.nml
ww3_prnc | tee ww3_prnc_wind.out
```

It interpolates a netCDF field onto your model grid and writes WW3's binary format (`wind.ww3`, `current.ww3`, `ice.ww3`, `level.ww3`). Key namelist fields:

```

&FORCING_NML
  FORCING%FIELD%WINDS = T
  FORCING%GRID%LATLON = T
  FORCING%TIMESTART   = 'YYYYMMDD HHMMSS'
  FORCING%TIMESTOP    = 'YYYYMMDD HHMMSS'
/
&FILE_NML
  FILE%FILENAME = 'gfs_winds.nc'
  FILE%LONGITUDE = 'longitude'
  FILE%LATITUDE = 'latitude'
  FILE%VAR(1)    = 'u10'
  FILE%VAR(2)    = 'v10'
/

```

5.4.1 The four things that go wrong

1. **Variable names.** u10 vs U10 vs 10u vs eastward_wind. ncdump -h first.
2. **Coordinate names.** A GRIB-derived file may carry time plus step rather than one valid-time axis, and ERA5 has shipped both time and valid_time across CDS format revisions. Look before you point the namelist at anything.
3. **Longitude convention.** 0–360 vs –180–180. GFS is served on 0–360, so the box above may come back as 308°–316° even though you asked for –52°...–44°; if your grid is at –52° and your forcing is at 308°, you may get a silent field of zeros rather than an error. Check with ncdump -v longitude gfs_winds.nc. If it needs shifting, NCO does it in place: get_gfs.sh ends with ncap2 -O -s 'where(longitude > 180.0) longitude = longitude - 360.0;' gfs_winds.nc gfs_winds.nc (nco 5.3.2 is in the pinned toolchain next to ncdump (v); the script refuses to run without ncap2).
4. **Coverage.** The forcing must cover the model domain *and* the model time window with a margin. WW3 will not extrapolate off the end of your winds; it will stop, or hold the last field, depending on version.

5.4.2 Verify before you run the model

Always output WND as a model field and look at it. If ww3_prnc silently produced nothing useful, a map of the wind WW3 *actually used* shows it in two seconds. This is the cheapest debugging habit in wave modelling. ncdump -v wnd ww3.nc | head is enough to tell zeros from weather.

5.5 Boundary spectra

Locally generated wind sea is only part of the picture. Swell arrives from thousands of kilometres away, and on many coasts it dominates.

```

parent run → ww3_ounp (spectral netCDF at points along the child's boundary)
           ↓
           ww3_bounc → nest.ww3 → read automatically by ww3_shel

```

ww3_bounc reads a `spec.list` file naming the spectral netCDF files to use:

```
ls ../configs/GLOBAL_30MIN/SPEC/ww3.*spec.nc > spec.list
ww3_bounc
```

There is no flag in `ww3_shel.nml` to enable boundaries: if `nest.ww3` exists in the run directory it is used. Check `log.ww3`: the input timeline shows boundary updates.

The alternative for global-scale sources: use NOAA's operational GFS-Wave output, or an IFREMER hindcast, as your parent. You don't have to run the global grid yourself.

5.6 Exercise

Take example 02, run it twice (once with boundary spectra, once without) and map the difference in H_s . The result is a map of "how much of the wave climate here comes from somewhere else". For the southern Brazil shelf in winter, it is most of it. `nccmp-tol` (lesson 06) gives you the max and RMS of that difference per field in one line; it will "fail", and the size of the failure is the answer.

→ [05-nesting.md](#)

Chapter 6

05. Nesting and multi-grid

Two ways to combine grids of different resolution. They are genuinely different, not two spellings of the same thing.

6.1 One-way nesting: chained `ww3_shel` runs

```
coarse run → ww3_oupn at points along the fine grid's boundary
           → ww3_bounc → nest.ww3
           → fine run
```

Simple, debuggable, embarrassingly parallel across the fine domains, and the standard approach when the parent is somebody else's operational output. The cost: no feedback. The fine grid cannot tell the coarse grid that a storm just generated a lot of energy.

Boundary setup lives in `ww3_grid.nml` for the *fine* grid:

```
&INBND_COUNT_NML
  INBND_COUNT%N_POINT = 2
/
&INBND_POINT_NML
  INBND_POINT(1) = 2  2  F
  INBND_POINT(2) = 2  80 T    ! CONNECT=T fills in everything between
/
```

Rules, both of which catch everyone:

- **Boundary points must be inside the grid.** Never on row/column 1 or N. The propagation stencil needs a cell outside them. This is the most frequently asked question on the WW3 Discussions board.
- `CONNECT = T` fills in every point on the straight line or diagonal from the previous point. Two entries define a whole edge.

Reference configuration: regtest `ww3_tp2.17`.

6.2 Two-way: `ww3_multi`

One executable runs a mosaic of grids simultaneously, exchanging spectra both directions each timestep. Driven by `ww3_multi.nml` instead of `ww3_shel.nml`, with one `mod_def.<grd>` per grid.

The grid definition line carries a rank and a group number. Ranks control which grid wins where they overlap; groups control which grids are computed concurrently. Getting this right is the actual skill in multi-grid WW3, and the reference material is the `mww3_test_01` through `mww3_test_08` regression tests plus the UFS documentation, which describes the `NFGRIDS` / `WINDLINE` / `ICELINE` / `CURRLINE` / `UNIPPOINTS` / `WW3GRIDLINE` structure.

`UNIPPOINTS` is worth knowing about: it gathers point output from all grids into one unified file, which otherwise is a genuine annoyance.

Two-way is what NOAA runs operationally. It is also substantially harder to debug, because a problem in one grid propagates into all of them.

6.3 Which to use

	One-way	Two-way
Parent is someone else's data	✓ only option	✗
Exploring / learning	✓	
Debugging	✓ much easier	
Storm generated <i>inside</i> the fine domain	✗ feedback lost	✓
Operational forecasting		✓
Many independent coastal domains	✓ trivially parallel	

Start one-way. Move to `ww3_multi` when you have a specific reason.

6.4 Unstructured as an alternative

A single unstructured mesh with 10 km elements offshore and 200 m in the inlet sidesteps nesting entirely. The trade: implicit schemes, PDLIB + ParMetis for parallel decomposition, and mesh generation becomes its own craft (OceanMesh2D, GMSH, SMS). Regtest `ww3_tp2.7` is the reference.

→ [06-output.md](#)

Chapter 7

06. Output and post-processing

7.1 The seven output types

Configured in `&OUTPUT_TYPE_NML` (what) and `&OUTPUT_DATE_NML` (when) in `ww3_shel.nml`. A stride of '0' disables a type.

1. **Gridded fields:** `Hs`, periods, direction, etc. on the model grid. → `out_grd.ww3` → `ww3_ounf` → netCDF. The one you'll use 90% of the time.
2. **Point spectra:** full 2D frequency-direction spectra at named locations. → `out_pnt.ww3` → `ww3_ounp` → netCDF. This is the model's actual state, not a summary.
3. **Track output:** fields along a moving track. For satellite collocation. → `ww3_trnc`.
4. **Restart files:** the complete model state, for continuing a run.
5. **Boundary data:** spectra along declared output boundaries, for feeding a child grid. Mostly superseded by using point output + `ww3_bounc` instead.
6. **Separated wave fields:** spatially and temporally coherent wave *systems* tracked across the domain. See the `ww3_syst rk` program and IFREMER's `TUTORIAL_WAVETRACK`.
7. **Coupling fields:** for NUOPC/ESMF/OASIS coupled runs.

7.2 Fields worth knowing

```
DPT CUR WND AST WLW ICE IBG D50 IC1 IC5
HS LM T02 T0M1 T01 FP DIR SPR DP HIG
EF TH1M STH1M TH2M STH2M WN
PHS PTP PLP PDIR PSPP PWS PDP PQP PPE PGW PSW PTM10 PT01 PT02 PEP TWS PNR
UST CHA CGE FAW TAW TWA WCC WCF WCH WCM FWS
SXY TWO BHD FOC TUS USS P2S USF P2L TWI FIC
ABR UBR BED FBB TBB
MSS MSC WL02 AXT AYT AXV
```

(the authoritative list with descriptions is at the top of `$WW3/model/nml/ww3_ounf.nml`)

Ones you'll actually reach for:

Field	What
HS	significant wave height, $4\sqrt{\text{total energy}}$
T01, T02, T0M1	mean periods from different spectral moments. They are not interch
FP, DP	peak frequency and peak direction
DIR, SPR	mean direction and directional spread
EF	the 1D frequency spectrum on the model grid
PHS PTP PDIR PSPR PWS	partitioned : wind sea + N swell systems, separately
USS, TUS	Stokes drift: what you hand to an ocean model
SXY	radiation stresses: what drives nearshore circulation
UST, CHA	friction velocity, Charnock: what you hand back to an atmosphere mod
WND, DPT	echo the inputs back. Free, and catches errors instantly.

7.3 Partitioning: the thing to actually understand

A total H_s of 2 m can be a 2 m wind sea, or a 2 m swell, or a 1.4 m sea plus 1.4 m of swell from a completely different direction. These are different oceans and different engineering problems, and a single number cannot distinguish them.

WW3 partitions the 2D spectrum into a wind sea and up to N0SWLL swell systems using a watershed algorithm on the spectral surface, and reports $PHS(n)$, $PTP(n)$, $PDIR(n)$ for each. `FIELD%PARTITION = '0 1 2 3'` in `ww3_ounf.nml` selects which to write (0 = wind sea).

Re-partitioning offline with different parameters, without rerunning the model, is something the third-party spectral libraries do well (lesson 08). In this repo the model's own partition is the one you get, and `ww3_ounp` gives you the full 2D spectrum if you want to do better by hand.

7.4 Reading the output

Everything `ww3_ounf` and `ww3_ounp` write is plain netCDF-4, and the tools that ship with netCDF itself are enough to read it. No library, no interpreter:

```
ncdump -h ww3.nc                # header: dimensions, variables, attributes, units
ncdump -v time ww3.nc | tail -5 # the time axis, decoded by hand from its units
ncdump -v hs ww3.nc | less      # the whole field, row-major, last time last
ncdump -h ww3.20240701_spec.nc  # the spectral file: (time, station, frequency, d
```

`ncdump` ships with `netcdf-c` and is in the pinned toolchain (v). For slicing, NCO's `ncks` is the tool: `ncks -v hs -d time,-1 ww3.nc` prints the last time step only, and `-d longitude,40 -d latitude,40` picks one point. `nco 5.3.2` is in the pinned toolchain alongside `cdo 2.5.1` (v), even though just `toolchain` does not print either of them.

Two habits worth forming:

- Read `:units` and `:scale_factor` before you read a number. `FIELD%TYPE` in `ww3_ounf.nml` is [2 = SHORT, 3 = it depends, 4 = REAL], template default 3 (v). With 2 the fields are packed short integers with a scale factor, and `ncdump` prints the raw

packed values, and it never unpacks (ncks --unpack does). Both course examples set `FIELD%TYPE = 4 (v)`: plain floats, and the problem goes away.

- The 2D spectrum is `efth(time, station, frequency, direction)`, in $\text{m}^2/\text{Hz}/\text{rad}$. H_s from it is $4 \sqrt{\sum \text{efth} \Delta f \Delta \theta}$. Computing that once from `ncdump` output (by hand or in twenty lines of Fortran) and checking it against the `HS` field is the consistency check that catches a wrong frequency range or a wrong $\Delta \theta$. If they disagree, one of them is being integrated over a different range than you think.

7.4.1 `ww_fetch_analyse`: the example-01 payoff

Example 01 is the one case with an analytic answer, and its analysis is a C++ program against `netcdf-c` rather than a notebook: [kokkos/tools/fetch_analyse/](https://github.com/kokkos/tools/blob/master/fetch_analyse/), built by just `kokkos-build openmp-release` and run by the example's `run.sh`.

```
ww_fetch_analyse ww3.nc [u10]          # u10 defaults to 10 m/s and must match HOMOG_INPUT(1)
```

It reads `hs(time, y, x)` (detecting `x` or longitude as the fetch axis), takes the last time step on the centre row, and prints one line per ~ 12 fetch bins:

Column	Meaning
fetch [km]	distance from the coastline at <code>i = 1</code>
WW3 H_s [m]	what the model produced at steady state
K&C92 H_s [m]	Kahma & Calkoen (1992) fetch law, $\hat{e} = 5.2e-7 \hat{x}^{0.9}$, converted to H_s
ratio	model / empirical: should sit near 1 while the sea is still growing

The header line prints the Pierson-Moskowitz fully developed limit, $H_s = 0.0246 U_{10}^2$ (2.46 m at 10 m/s); nothing should meaningfully exceed it at steady state, and a ratio that drifts *down* with fetch is the sea approaching full development, not an error. An H_s that *decreases* with fetch means your wind direction convention is flipped.

7.4.2 Compare two runs with `nccmp-tol`

The second half of the course is built on one question: *did this change alter the answer?* Compile flags, an OpenMP layout, a refactored routine, a Kokkos kernel: every rung of the ladder is gated by that question, and “eyeball two `ncdumps`” is not an answer. [kokkos/tools/nccmp-tol/](https://github.com/kokkos/tools/blob/master/nccmp-tol/) is the comparator the proposal calls “comparador por campo”:

```
nccmp-tol REF.nc TEST.nc [TOLERANCES]          # default: kokkos/tools/nccmp-tol/tolerances.tx
```

Its contract:

- Every numeric variable present in **both** files is compared over the values that are neither NaN nor `_FillValue`; per variable it prints `n`, `max_abs`, `rms` and `max_rel`.

- A variable is **judged** only if it is listed in the tolerances file, one line each: name abs rel, # comments allowed. A value passes if $|d| \leq \text{abs}$ **or** $|d| / \max(|\text{ref}|, \text{eps}) \leq \text{rel}$. Unlisted variables are reported, not judged.
- Exit **0** only if every judged variable passes; **1** if any fails; **2** on an I/O error. So it goes straight into a shell if, a CI step, or L2_replay.sh (lesson 12).

The default file judges `hs`, `fp`, `dir`, `dp` and `t0m1`: $1e-4$ relative everywhere, $1e-4$ absolute on the scalar fields and $1e-2$ absolute on the two directions (they are in degrees). Those numbers are a starting point for a bit-reproducible change, not a scientific statement; the proposal's rule is that the tolerances live in a versioned file, are proposed by the student and approved by the co-advisor, and a change that fails them is not merged. Tightening or loosening them is a commit with a reason, not a command-line flag.

Two things it is *not*: it is not `nccmp` (the C tool of that name compares bit for bit and knows nothing about tolerances), and it is not a validator: it compares a run against another run, never against the sea.

7.5 Validation

[NOAA-EMC/ww3-tools](#) is the official toolkit: altimeter collocation, NDBC buoy matching, scatter plots, QQ plots, Taylor diagrams, and the standard metric set. It is Python, and it is the right tool for that job (lesson 08).

Rough expectations for a regional run with default tuning and decent winds: `Hs` bias within $\pm 10\%$, scatter index 15–25%. Periods are worse: `Tp` in particular is a noisy statistic and comparing it point-to-point against a buoy is a good way to feel bad about yourself. Compare `T0M1` instead, and compare *distributions* as well as time series.

If your first attempt looks dramatically better than that, check that you aren't accidentally comparing the model against itself.

→ [07-physics-choices.md](#)

Chapter 8

07. Physics choices: which source terms, and why it matters

Everything here is a **compile-time** choice in the switch file. Changing it means a full rebuild. See [../switches/README.md](https://github.com/NOAA-EMC/ww3/blob/master/switches/README.md).

8.1 The source-term packages

Package	Origin	Character
ST0	none	Wind input and dissipation off entirely. For pure propagation.
ST1	WAM Cycle 3	Historical. Don't.
ST2	Tolman & Chalikov (1996)	The old WW3 default. Still in use operationally at some places.
ST3	WAM Cycle 4 / Bidlot	ECMWF lineage.
ST4	Ardhuin et al. (2010)	Saturation-based dissipation plus an explicit swell-dissipation term.
ST6	Rogers, Babanin, Zieger	Built from direct field observations of wind input and wind-wave dissipation.

Pick one. They are **matched sets**: the input and dissipation terms in a package are tuned against each other, and mixing them produces nonsense.

8.2 Why this matters more than you'd like

S_{in} and S_{ds} are each large, each uncertain at the tens-of-percent level, and they nearly cancel. The evolution of the spectrum is the small residual. Two defensible modern packages, run on the same grid with the same winds, can differ by 10–20% in H_s and more in period.

This is not a failure of WW3. It is the honest current state of wind-wave physics. The practical implication: **your model has a tuning, and you should know what it is**. Running ST4 and ST6 on the same case and comparing is one of the most educational things you can do, and `examples/01` is set up for exactly that.

8.3 The nonlinear term

NL1, the Discrete Interaction Approximation (Hasselmann et al. 1985), is what almost everyone runs. It approximates the four-wave resonant interaction integral with a single representative quadruplet configuration.

- NL2 computes it (nearly) exactly, at roughly 1000× the cost. Research only.
- NL3 is the Generalized Multiple DIA: several quadruplets with optimised coefficients. Better, more expensive, and the coefficients need fitting: [genes_gmd](#) exists for that.

NL1 is simultaneously the most important and the most approximate term in the model. Some of what ST4/ST6 tuning is compensating for is DIA error, not real physics. Worth knowing when you're tempted to interpret a tuning constant physically.

8.4 Propagation

PR3 UQ, third-order ULTIMATE QUICKEST with the Garden Sprinkler Effect correction, is the default and the right answer for almost everything. Alternatives: PR1 first-order upwind (very diffusive, but rock stable), PR2 second-order, PR0 no propagation.

The **Garden Sprinkler Effect** is worth understanding: with a discrete direction grid, each bin propagates as an independent ray, so a swell field far from its source disintegrates into separate beams. It is a pure discretisation artefact. PR3's correction diffuses across directions to suppress it. More direction bins also helps. If you see striping in a swell field far from the storm, this is why.

8.5 Shallow water

Switch	Physics
BT1	JONSWAP empirical bottom friction. Fine by default.
BT4	SHOWEX movable-bed friction. Needs a D50 sediment map (&SED_NML, &SBT4 SEDMAPD50
DB1	Battjes-Janssen depth-induced breaking.
MLIM	Miche-style limiter on Hs in shallow water.
TR0/TR1	Triad (three-wave) interactions: surf-zone energy transfer to harmonics.
REF1	Shoreline reflection. Needs a slope map (&SLOPE_NML, &REF1 REFMAP = 2).

If your domain is entirely in deep water, none of this fires and you can leave the defaults.

8.6 Sea ice

IC0 (off) through IC5, plus IS1/IS2 for scattering. IC5 (the extended Fox-Squire model) arrived in v6.06. The ice forcing fields come in through INPUT%FORCING%ICE_CONC and ICE_PARAM1...5 (thickness, floe size, viscosity ...).

Marginal-ice-zone wave physics is an active research area and the parameterisations disagree with each other substantially more than the open-water ones do.

8.7 Tuning namelists

Physics constants live in the file pointed at by GRID%MML (conventionally namelists.nml), read by ww3_grid:

```
&SIN4
  BETAMAX = 1.52      ! the master gain on wind input. Bump this, everything gets bigger.
/
&SDS4
  SDSBCHOICE = 2
/
&MISC
  FLAGTR = 0
/
```

ww3_grid writes the namelists it actually used into param.scratch and echoes them to stdout. Check that file: it tells you what the model is really running, as opposed to what you thought you configured. Blocks irrelevant to your switch settings are silently skipped, which means a typo'd namelist name fails quietly.

8.8 A note on tuning

The temptation, once you've validated against a buoy and found a bias, is to reach for BETAMAX. Resist it until you've checked, in this order:

1. Your winds (the dominant error source, usually by a wide margin)
2. Your bathymetry and obstruction grids
3. Your boundary conditions / missing swell
4. Your spectral range: is FREQ1 low enough for the swell that's actually there?
5. Your resolution
6. *Then* the physics constants

Tuning a source term to compensate for a bad wind field produces a model that's right for the wrong reason and will fail on the next storm.

8.9 Why W3SNL1 is the first kernel to port

Everything above is about choosing physics. The second half of the course is about making the chosen physics run faster, and the first routine it rewrites is the one this lesson called the most important and the most approximate: W3SNL1, the DIA, with its setup routine INSNL1 (both in `model/src/w3snl1md.F90`). Five properties make it the right first target, and they are worth naming because they are the checklist for every routine after it:

Property	What it means for a port
Per point	One spectrum in, S and D out, no neighbours. Every sea point is
Table-driven	INSNL1 precomputes the quadruplet address tables (IP11 ... IM42)
No I/O, no globals mutated	It reads a handful of grid constants and writes its two outputs.
Dominant share	Usually the single most expensive kernel in a ST4+NL1 run: four
Deterministic gather	Each output bin is a weighted <i>gather</i> from the extended spectr

The trade-off is instructive too: the DIA is a physics approximation everybody wants to replace, so why port it? Because the port is a *translation*, not an improvement. The Kokkos kernel must reproduce the Fortran to round-off before anyone is allowed to touch the physics, and a routine with an exact, cheap, per-point reference is the easiest one to prove that claim on. NL2 and NL3 would inherit the same kernel structure later. [12-porting-a-kernel-w3snl1.md](#) walks through the port line by line; the code is in `kokkos/src/ww_kokkos/sn11_*` with the verbatim Fortran reference beside it in `kokkos/tests/fixtures/sn11_ref.F90`.

→ [08-python.md](#): the Python ecosystem, and why this repo does not depend on it.

Chapter 9

08. The Python ecosystem, and why this repo does not depend on it

There is a real Python ecosystem around WW3, and you should know what is in it. This repo's rule: the lab code in `kokkos/`, `examples/`, `exercises/` and `bench/` is C++, Fortran and shell; Python is used only by the two publishing scripts in `scripts/` and their unit tests in `tests/`. What the tools do, what one taught us, what we use instead:

9.1 The landscape

Tool	Role
pyww3	Generate namelists from typed dataclasses and run the WW3 executables. The
WW3-tools	NOAA's official post-processing and validation: altimeter, buoys, statistics, plo
wavespectra	The xarray library for spectral data. Reading, partitioning, statistics, polar plo
ww3tool	More ambitious wrapper: namelists + run scripts + Slurm submission + plott
bmi-wavewatch3	Download NOAA's <i>published</i> WW3 hindcasts as xarray. Not for running the m
rompy	General pydantic-validated ocean-model configuration. Strong SWAN/SCHISM

9.2 What pyww3 gets right

Its design is worth holding in your head even if you never install it. Every WW3 program gets a dataclass (`WW3Grid`, `WW3Prnc`, `WW3Shel`, `WW3Ounf`, `WW3Ounp`, `WW3Bounc`); every namelist parameter becomes a constructor keyword with the % flattened to `_`, so `SPECTRUM%FREQ1` is `spectrum_freq1`; validation of required files and compatible values happens at construction; and a common base class renders the namelist text, writes it into the run directory, runs the program there capturing its output, and can add or remove a whole namelist block, needed because `ww3_grid` objects to blocks it does not want. Its two gotchas are the two lessons: the generated file contains *every* block until you remove the irrelevant ones, and after you mutate an attribute you must regenerate the text or the file on disk keeps the old value. The

author's own words: *work in progress, API not stable*. Being made to name every parameter is what teaches you the namelist, and once you understand it, writing your own generator for what you actually need is a two-hour job. That is the job the Fortran generators in `examples/` do.

9.3 Why this repo does not depend on any of it

The second half of the course is a benchmarking and porting project, and its rule is that every number must be reproducible by the lab on its own machines from one pinned toolchain (lesson 01). A Python *dependency* is a second toolchain, with its own resolver and its own drift; the day it breaks is the day you cannot rerun last month's benchmark. (Availability is another matter: the pinned `#ww3` shell ships python3 with numpy and xarray (v); nothing in the lab code imports them.) WW4 did the same: it merged "Remove python dependence from compile system" on 2026-08-19 (v) (lesson 14). And the lab code has to be *inside* the parity gate: a generator, analyser or comparator outside the compiled, tested tree is where a silent change hides. None of that is a criticism of the tools above: WW3-tools remains the right way to validate against buoys and altimeters, and wavespectra the best way to re-partition a spectrum offline.

9.4 What we use instead

Need	Python route	What this repo
Namelists	pyww3 dataclasses, Jinja2 templates	By hand from t
Parameter sweeps and benchmark cases	scripted loops over pyww3 objects	ww_bench_case
Forcing download	cdsapi for ERA5	get_gfs.sh + ec
Reading output	xarray, wavespectra	ncdump (netcdf-
"Did the answer change?"	ad-hoc numpy.allclose	nccmp-tol REF "
Per-routine tests	none	GoogleTest aga

→ [09-benchmark-profile-compile-run.md](#): measure first.

Chapter 10

09. Measure first: benchmark, profile, compile options, run configuration

This lesson is steps 1 and 2 of the proposal's ladder (pubs/proposal/pt/04-scope.md): everything you can do to a WW3 installation **without touching a line of Fortran**. It is also where the instrument for every later lesson is built. Nothing in 10-13 means anything without a reference run, a metric and a profile.

10.1 The reference run

Freeze five things and write them down: the code revision (`git -C $WW3 rev-parse HEAD`), the switch file, every namelist, the grid and the forcing. The proposal calls this the *execução de referência congelada*; every number in the course is relative to it.

The lab's smallest reference is an upstream regtest run step by step (v):

```
just rt ww3_tp1.1 PR3_UQ      # build with the test's own switch_PR3_UQ, then run it
```

`just rt` chains `just build` and `just regtest`; the latter is `scripts/03_run_regtest.sh`, which copies `regtests/<test>/input/` into `regtests/<test>/work_lab/` and runs `ww3_grid`, `ww3_strt`, `ww3_prnc`, `ww3_bounc`, `ww3_shel`, `ww3_ounf`, `ww3_ounp` in order, each only if its `.nml/.inp` is present (v). It deliberately does not use `matrix.comp`: the point is to watch the pipeline. The operational case at LabECO gets the same treatment once the lab captures it.

10.2 The metric

Wall-clock seconds per forecast hour of `ww3_shel`. Not CPU time, not the whole pipeline. `bench/README.md` lists the traps (v): `CPU_TIME` sums across OpenMP threads and makes threading look like a slowdown, so use `SYSTEM_CLOCK`; `ww3_grid` is serial setup and is excluded; field output is set to stride '0' so you measure compute, not the filesystem; run more than once and report medians because both an i9 and a

4090 clock down under sustained load. If you want reproducible numbers, lock the clocks (bench/README.md has the `nvidia-smi -lgc / cpupower` lines).

10.3 bench/

What	Program	Measures
The real WW3, CPU only	<code>bench/bench_ww3_cpu.sh <build> [case] [max-ranks]</code>	wall time of ww
A WW3-shaped kernel	<code>bench/kernel_bench.f90</code>	CPU vs GPU, d
Everything	<code>bench/run_all.sh [build]</code>	the kernel pro

The case itself comes from the C++ generator: `ww_bench_case --size small|medium|large` (or `--nx --ny --nk --nth --hours`) -o DIR writes `depth.inp`, `mask.inp`, the three namelists and a `case.json`, with the timesteps derived from the CFL condition so a bigger grid stays stable and does the work it claims to do. `just bench` builds it in `kokkos/build/openmp-release` first. Requires a build with switches/switch_lab_mpi (DIST MPI) (v).

Read `bench_ww3_cpu.sh`'s closing note before believing the curve (v): on an i9, P-cores and E-cores differ so much that an MPI job runs at the pace of its slowest rank. Rerun pinned (`mpirun -np 8 --bind-to core --cpu-set 0-15`) and compare; eight P-cores beating 24 mixed cores is normal for a memory-bound spectral model. If efficiency is flat from rank 1, the case is too small: use a larger `--size`.

10.4 Profile

```
just profile ww3_tp1.1          # kokkos/tools/profile/gprof_table.sh <ww3-dir> <regtest>
```

`gprof_table.sh` rebuilds WW3 with `-pg`, runs the `regtest`, and turns `gprof -b` into a table routine `| self % | cumulative %` for the top 25 routines, grouped by phase with a name map: `w3srce*` → source terms, `w3pro*/w3uqck*` → propagation, `w3gath/w3scat/mpi_` → communication, `w3io*` → I/O. `perf_table.sh` produces the same table from `perf record -g` where `perf` exists (it is not in the `pratico` shell). `exercises/solutions/ex10_profile.sh` is the worked version.

What to expect, as priors only (`docs/AGENTS_KOKKOS_202609.md` §2.1, $\triangle$ not measured here):

Bucket	WW3 modules
Source terms	<code>w3srcemd</code> (W3SRCE), <code>w3snl1md</code> , <code>w3src4md/w3src6md</code> , <code>w3sbt*</code> , <code>w3sdb1m</code>
Spatial propagation	<code>w3pro2md/w3pro3md</code> , <code>w3uqckmd</code>
Intra-spectral propagation	<code>W3KTP2/3</code>
Gather/scatter transposes, MPI	<code>W3GATH/W3SCAT</code>
Output and restart I/O	<code>w3iogomd</code> , <code>w3iorsmd</code> , <code>w3iopomd</code>

Profile at 1, 4 and 16 ranks (the proposal's methodology). The communication share is the number that changes, and it is the argument for or against more ranks.

10.5 WW3's own matrix: the bit-for-bit gate

WW3 ships 62 regression cases under `regtests/` (v, counted in the clone at `~/src/WW3`). `regtests/bin/matrix.base` generates the run script from a list of options that includes `rstrt_b4b`, `nth_b4b` and `npl_b4b` (v): restart, thread-count and MPI-task-count reproducibility variants, each of which runs a case twice and demands identical output. `regtests/bin/matrix.comp` compares two matrix runs file by file with `cmp` for binaries and `diff` for text, and sorts every case into "identical" or "non-identical" (v). There is no tolerance anywhere in it: any numerical difference is a failure that a human then judges. Unit tests exist since 2023 and cover only I/O (`pubs/proposal/pt/05-justificati`

That is exactly the right gate for steps 1 and 2. A compiler flag, a switch or a namelist change that alters no floating-point *ordering* must reproduce the reference bit for bit, and the matrix will tell you. A change that reorders operations (vectorisation width, `-Ofast`, a different MPI decomposition of a reduction) is the case for the next section.

10.6 Step 1: the compile-option matrix

Vary one axis at a time. Everything here is baked in at build time (`scripts/02_build_ww3.sh` wipes build/ on every run (v)).

Axis	Values worth trying
Compiler	gfortran 15.3 (pinned in <code>pratico</code> (v)); nvfortran on the CPU as a stepping stone
Flags	-DCMAKE_BUILD_TYPE=Release vs Debug; -O2 vs -O3 -march=native
Parallel switch	SHRD (serial), DIST MPI, plus OMPG/OMPH for OpenMP on top (v switches/README.md)
Output	FILE%NETCDF 3 vs 4 in <code>ww3_ounf.nml</code> and the output stride; NC4 in the switch file

`exercises/solutions/ex09_matrix.sh` runs three of these axes on `ww3_tp1.1` and tabulates wall-clock and the comparator's verdict; `exercises/ex09_bench.md` is the sheet.

10.7 MPI × OpenMP on one node

With `switch_lab_mpi` you have ranks; with OMPG OMPH added you have threads inside ranks. On one workstation the layout question is: how many ranks, how many threads each, pinned where.

Layout	When it wins
Pure MPI, one rank per physical core, pinned	almost always on a workstation; WW3's "card c
Fewer ranks, threads inside	when the rank count is so high that the gather/
Ranks on hyperthreads	rarely; test one rank per physical core against

Set `OMP_NUM_THREADS` explicitly, set `OMP_PROC_BIND` explicitly (Kokkos warns if you do not (v `kokkos/CMakeLists.txt`)), and always pass `--bind-to core`. Then run the `npl_b4b/nth_b4b` variants: the layout must not change the answer.

10.8 Step 2: run-configuration knobs, one at a time

These are namelist edits only. Change one, rerun, compare, keep or revert.

Knob	Where	Notes
TIMESTEPS%DTMAX, DTX, DTKTH, DTMIN	ww3_grid.nml (v)	the templ
Output frequency and field list	ww3_shel.nml: TYPE%FIELD%LIST, *%TIMESTRIDE (v)	the fastest
Restart stride	ww3_shel.nml restart block	rstrt_b4b
Forcing interpolation	WNT1 WNX1 / CRT1 CRX1 in the switch (v)	compile-t

10.9 When reordering breaks b4b: nccmp-tol

```
nccmp-tol REF.nc TEST.nc # default kokkos/tools/nccmp-tol/tolerances.t
nccmp-tol REF.nc TEST.nc my_tolerances.txt # just nccmp does the same
```

For every variable listed in the tolerances file (rows `hs`, `fp`, `dir`, `dp`, `t0m1` by default; format name `abs rel`) it prints max-abs, RMS and max-relative differences over non-fill values and exits 0 only if every judged variable passes. Unlisted variables are reported, not judged. The tolerances file is versioned and approved by the co-advisor: that is the proposal's *comparador por campo*, and it is what lessons 10–13 use once the matrix's yes/no answer is no longer the right question.

The GPU question, short version. Yes, NVIDIA ships a production Fortran compiler: `nvfortran` in the free HPC SDK, with OpenACC (`-acc -gpu=cc89`), OpenMP target (`-mp=gpu`), CUDA Fortran (`-cuda`) and do concurrent offload (`-stdpar=gpu`). An RTX 4090 is compute capability 8.9, so `cc89`, and a consumer card is fine. But upstream WW3 has no GPU code path at all, and the one serious attempt, Ikuyajolu et al. (2023), *GMD* 16, W3SRCEMD under OpenACC on V100 nodes, reached about **1.3× against 42 CPU cores**, limited by host↔device transfer and by register pressure from the routine's many locals; it is not merged upstream $\triangle$ check. Your PCIe 4.0 link is slower than the NVLink they had, and a 4090 runs FP64 at 1/64 of FP32. So: measure the CPU first (this lesson), and when you do go to the GPU, do it with kernels that own their data (lessons 11, 12, 13), not with directives sprinkled on Fortran (lesson 10 shows why).

10.10 Sources

- Ikuyajolu et al. (2023), *GMD* 16, 1445–1458: <https://doi.org/10.5194/gmd-16-1445-2023>
- `pubs/proposal/pt/05-justification.md`, “Estado dos testes do WW3” (survey of 2026-09-15)
- `docs/AGENTS_KOKKOS_202609.md` §2.1, §2.4 (the profiling recipe)

→ [10-modern-fortran-refactoring.md](#)

Chapter 11

10. Modern Fortran refactoring: the step before any port

Step 3 of the ladder. The profile from [lesson 09](#) names the routines; this lesson rewrites them in place (same algorithm, same arithmetic, standard Fortran 2008/2018) and proves per routine that nothing changed. It is worth doing on its own (the gain is measured on the model the lab runs), and it is what makes a routine portable at all: a routine with an explicit interface and contiguous data can be replaced by a kernel; one that reads module state through five USE lines cannot.

11.1 What a profile-top routine looks like

W3SNL1 (WW3/model/src/w3snl1md.F90, lines 115–473 in 7.14 (v)) is the DIA source term and the first thing the lab ported. Its interface is already good:

```
REAL, INTENT(IN)      :: A(NSPEC), CG(NK), KDMEAN
REAL, INTENT(OUT)     :: S(NSPEC), D(NSPEC)
```

Its workspace is not. Every call declares, sized at run time from module variables (v, kokkos/tests/fixtures/snl1_ref.F90 lines 335–339, verbatim from WW3 lines 324–328):

```
REAL      :: UE (1-NTH:NSPECY), SA1 (1-NTH:NSPECX), &
SA2 (1-NTH:NSPECX), DA1C(1-NTH:NSPECX), &
DA1P(1-NTH:NSPECX), DA1M(1-NTH:NSPECX), &
DA2C(1-NTH:NSPECX), DA2P(1-NTH:NSPECX), &
DA2M(1-NTH:NSPECX), CON (      NSPEC )
```

Ten **automatic arrays**, one of them with a lower bound of 1-NTH so that clamped quadruplet addresses can read zeros. NK, NTH, SIG, the 32 address tables IP11..IC82, the weights AWG1..8 and AF11 all arrive through USE W3GDATMD / USE W3ADATMD (v). Test output and tracing sit behind `#ifdef W3_T / W3_S` (v). This is the shape of most of WW3's inner routines, and W3SRCE is the same shape with dozens of locals and 239 preprocessor guards in its file (docs/KOKKOS_H100_PLAN_202609.md, $\triangle$ counted there, not here).

The properties that matter: per point, no I/O, reads shared tables, writes two arrays. Everything in this lesson is about making those properties *visible in the interface*.

11.2 The checklist

Construct	What it buys
Module procedure (explicit interface)	the compiler checks every call; a kernel can read the data-flow contract a shim needs
INTENT(IN/OUT/INOUT) on every argument	no copy-in on the call, a pointer that C can take
CONTIGUOUS on assumed-shape dummies	no hidden I/O or global writes; required for DO
PURE	no automatic arrays, no per-call stack traffic
Workspace by argument or per-grid allocation	the routine can be called from a test with capture
Tables passed in, not USED	

Do them in that order. The first two cost nothing in WW3 because the code is already modular; the last two are the work.

11.3 DO CONCURRENT

gpu/02_do_concurrent.f90 is the example: the dispersion relation solved by Newton at every (point, frequency), in standard Fortran with no directives (v):

```
do concurrent (ip = 1:npt, ik = 1:nk) local(x, kd, f, fp, t, it)
```

One source, three targets: `nvfortran -stdpar=gpu -gpu=cc89`, `nvfortran -stdpar=multicore`, plain `gfortran -O3` (v, header of that file). The file's own caveats (v): `LOCAL()` is Fortran 2018 locality, and if a compiler rejects it you may drop the clause, but then every scalar assigned inside the loop is one the compiler has to prove private on its own, and `-Minfo=stdpar` must be read to see whether it did or whether the loop went serial. Also `-stdpar=gpu` moves allocatables to CUDA managed memory, so data movement becomes implicit: convenient, and exactly how a naive port ends up thrashing PCIe without noticing $\triangle$ (the file says so; not measured here).

The construct is honest about one thing the directives are not: the loop body must be independent per iteration and free of side effects. Writing W3SNL1's section 3 as a `DO CONCURRENT` over ISP is legal; writing its section 2 tail loop that way is not, because row IFR reads row IFR-1 (v, `snl1_ref.F90` `UE(ISP) = UE(ISP-NTH) * FACHFE`). The port in lesson 12 keeps that loop sequential for the same reason.

11.4 Killing automatic arrays

Three ways, in increasing distance from the original:

1. **Allocate once per grid.** WW3 already does this for the tables: INSNL1 section 6 calls W3DMNL, which allocates IP11..AF11 in W3ADATMD once (v, `snl1_ref.F90`)

comment on section 6). Do the same for UE, SA1...: module-level ALLOCATABLE workspace sized 1-NTH:NSPECY at grid setup. Cost: thread-private copies under OpenMP.

2. **Pass workspace in.** Add a WORK derived type or explicit dummies. The routine becomes testable in isolation and the caller decides who owns the memory.
3. **Team scratch on the device.** The Kokkos version gives each sea point its own scratch and offsets every index by +NTH so the 1-NTH lower bound becomes slot 0 (v, kokkos/src/ww_kokkos/sn11_dia.cpp). Lesson 12.

Whichever you pick, the arithmetic inside the loops does not change. That is the rule the parity test enforces.

11.5 The runtime switch: one binary, two paths

The proposal's method for step 3 (pubs/proposal/pt/07-methodology.md): the refactored routine lives next to the original in a fork, and a switch read once at start-up picks one. The regression matrix and nccmp-tol then run both paths without a rebuild.

The lab's Kokkos interface module shows the pattern (v, kokkos/src/fortran_iface/w3kokkosmd.F90)

```
LOGICAL :: KOKKOS_SNL1 = .FALSE.
...
SUBROUTINE W3KOKKOS_SETUP
  KOKKOS_SNL1 = ( WW_SNL1_ENABLED() == 1_C_INT )
END SUBROUTINE W3KOKKOS_SETUP
```

WW_SNL1_ENABLED() reports whether WW_KOKKOS_SNL1=1 was in the environment when the runtime started (v, ww_kokkos_c.hpp). The model reads a LOGICAL in its inner loop, not an environment variable, and not a C call per sea point. A Fortran-only refactor uses the identical shape: W3SNL1_NEW beside W3SNL1, one logical, IF (...) THEN CALL ... ELSE CALL ... END IF at the single call site. Per-call branches inside kernels are the anti-pattern (docs/AGENTS_KOKKOS_202609.md §1.6).

11.6 The per-routine parity test

The matrix compares whole runs. Step 3 needs something finer: run the original and the rewrite on the *same captured inputs* and compare the outputs.

exercises/solutions/ex11_refactor.F90 holds a 40-line WW3-style routine (automatic array, implicit interface) and its modern twin; exercises/solutions/ex11_refactor_test.F90 drives both on random input and stops with a non-zero status above 1e-6, built with plain gfortran. That is the small version.

The full-size version is the fixture generator the port uses (v, kokkos/tests/fixtures/gen_sn11_fix). It call the verbatim reference on a deterministic sea state, stream inputs, tables and outputs into a little-endian file with ACCESS='STREAM', refuse to write if the default REAL is not 32-bit, commit the file. Any rewrite, Fortran or C++, is then tested against the same bytes. Same arithmetic should give the same bits; 1e-6 relative

is the room you leave for a compiler that reassociates a sum. If you need more room than that, you changed the algorithm, and that is a separate change with its own evidence.

11.7 The contrast: directives

`gpu/00_hello_acc.f90` is the smallest OpenACC program that proves the toolchain (v): `!$acc data create(a, b)`, two `!$acc parallel` loops, a reduction(`+:s`), and the advice that if `NVCOMPILER_ACC_NOTIFY=1` prints no launch line you built a CPU binary. Reading `-Minfo=accel` is most of OpenACC development.

Ikuyajolu et al. (2023) did this to `W3SRCEMD`: OpenACC on the source-term driver, MPI for the rest, V100 nodes with NVLink. Result: about $1.3\times$ against 42 CPU cores, and the paper is explicit about why (lesson 09's aside): the state lives in Fortran module arrays on the host, so every call ships the spectrum across the bus, and a routine with that many locals starves the GPU of occupancy. Directives change *where a loop runs*; they do not change *who owns the data*. That ownership question is the whole content of lesson 11: a View names its memory space, a kernel can only touch what it was handed, and the residency ladder in lesson 13 is the plan for moving the state, one array at a time.

11.8 Checklist for one routine

1. Profile says it matters (lesson 09). If not, stop.
2. Capture its inputs and outputs with a generator like `gen_sn11_fixture.F90`.
3. Explicit interface, `INTENT`, tables as arguments, workspace out of the body.
4. Runtime switch, both paths in the binary.
5. Parity test on the capture; then the matrix on the closest regtest; then `nccmp-tol` on the operational case.
6. Re-profile. The residual is what lesson 12 is allowed to port.

→ [11-kokkos-and-modern-cpp.md](#)

Chapter 12

11. Kokkos and modern C++ for Fortran people

Kokkos is a C++ library that lets one kernel run on a CPU thread pool or a GPU without rewriting it: the backend is a build choice. Every concept below is a Fortran concept with a new name, and every one has a runnable program in kokkos/intro/ that checks itself and exits non-zero if the concept did not hold (v, kokkos/intro/CMakeLists.txt). Read each program next to its section. Build them first:

```
just kokkos-test serial-debug      # configure + build + ctest, inside the pratico #ww3 shell
```

Fortran	Kokkos	Pro
REAL, ALLOCATABLE :: A(:, :)	Kokkos::View<Real**, DeviceSpace> a("label", n, m)	01_
column-major order	Kokkos::LayoutLeft	04_
DO loop over independent iterations	parallel_for with a RangePolicy / MDRangePolicy	02_
SUM, MAXLOC, prefix sum	parallel_reduce with a reducer, parallel_scan	03_
automatic array per point	TeamPolicy + team_scratch(0)	05_
BIND(C) interface block	extern "C" + unmanaged View	06_

12.1 Execution and memory spaces

A kernel runs in an **execution space** (Serial, OpenMP, CUDA) and touches memory in a **memory space** (HostSpace, CudaSpace). The lab spells the memory space out on every View (v, every intro program):

```
using DeviceSpace = Kokkos::DefaultExecutionSpace::memory_space;
```

That alias is HostSpace in the #ww3 shell and CudaSpace in #cuda, so the same source compiles everywhere, but the declaration now says whether host code may dereference the array. Leaving it implicit is how host code ends up reading device memory (v, comment in 01_views.cpp). The backend comes from the *shell*, not the preset: OpenMP is the default execution space in #ww3, so serial-debug is a build type, and code that must run serially names Kokkos::Serial explicitly (v, kokkos/README.md).

12.2 View

01_views.cpp (v): Kokkos::View<ww::Real**, DeviceSpace> spectrum("intro.spectrum", nth, nk). The string is a label that shows up in profilers and bounds-check aborts, so spell it. rank(), extent(i) and span_is_contiguous() are carried by the View, so no separate nth, nk arguments travel with the array. A fresh View is zero-initialised unless you ask for Kokkos::WithoutInitializing. Views are reference-counted handles: copying one copies a pointer, and the allocation dies with the last handle, which must happen *before* Kokkos::finalize(), hence the inner scope in every intro main (v). ww::Real is float (v, kokkos/src/ww_kokkos/real.hpp).

12.3 Layout: LayoutLeft is Fortran order

Layout is part of the type and decides which index is contiguous. 04_layouts_and_mirrors.cpp asserts it (v): LayoutLeft has stride(0) == 1, stride(1) == nth; LayoutRight the reverse. Kokkos defaults to LayoutRight on host spaces and LayoutLeft on CUDA because each coalesces on its backend. You override it in exactly one place: the Fortran boundary. A WW3 array A(NTH,NK) wrapped as anything but LayoutLeft still "works": every index is silently transposed. deep_copy between different layouts is a transpose, correct and expensive, which is the argument for choosing once at the boundary rather than converting inside a time step (v).

12.4 Mirrors and deep_copy

You may not dereference a device View from host code. create_mirror_view(v) gives a host View with the same layout; deep_copy(host, v) fills it. On a host backend the mirror is the original and the copy is a no-op, so the pattern costs nothing when it is not needed (v, 01_views.cpp). create_mirror_view_and_copy(Kokkos::HostSpace(), v) is the one-liner the tests use (v, 03_reduce_and_scan.cpp).

12.5 parallel_for, parallel_reduce, parallel_scan

02_parallel_for.cpp builds a JONSWAP spectrum over a WW3-shaped grid with MDRangePolicy<Rank<2>>({0,0},{nth,nk}) and integrates it back to Hs with a parallel_reduce, checked against the closed-form m0 to within 2 % (v). Two habits in that file: the reduction accumulator is double even though the state is float, because 768 terms over five decades lose digits otherwise; and the check against something computed independently *is* the point: a kernel you cannot compare is one you cannot port.

03_reduce_and_scan.cpp (v): Kokkos::MaxLoc<Real,int> returns the largest Hs *and* where; the reducer, not the lambda, owns the identity and the join, so it is right on every backend. parallel_scan calls the functor more than once and only the is_final pass may write. A WW_DETERMINISTIC build pins the scan to Kokkos::Serial after asking SpaceAccessibility whether Serial can even reach the device memory (v).

12.6 TeamPolicy and scratch

05_team_scratch.cpp is the shape of the DIA kernel (v): TeamPolicy(npts, Kokkos::AUTO) makes one **team** per sea point; policy.set_scratch_size(0, PerTeam(bytes)) with bytes = ScratchSpectrum::shmem_size(nth, nkx) reserves level-0 scratch: shared memory on a GPU, a slice of a thread-local arena on a CPU; inside the lambda, ScratchSpectrum ue(team.team_scratch(0), nth, nkx) is an *unmanaged* view onto it (scratch is a bump allocator; nothing frees). TeamThreadRange(team, n) parallelises across the team, team.team_barrier() separates writing scratch from reading it (not optional), and Kokkos::single(PerTeam(team), ...) lets one thread write the team's result. The extended spectrum with its parametric tail lives there because it is per point, re-read many times, and must never be a global allocation inside a time loop (v). The port's scratch for NK=25, NTH=24 is about 28 KB, well inside TeamPolicy::scratch_size_max(0), which for the Kokkos 5.2 CUDA backend is the device's opt-in shared-memory limit minus ~24.6 KB: about 75 KB on the RTX 4090, not the classic 48 KB (v, snl1_dia.cpp, Kokkos_Cuda_Parallel_Team.hpp).

12.7 KOKKOS_LAMBDA and what a kernel may touch

KOKKOS_LAMBDA captures by value. A View is a handle, so capturing it is cheap and correct; snl1_dia.cpp copies its Tables struct into a local before the lambda for exactly that reason (v). The rule ("no host memory in kernels", docs/AGENTS_KOKKOS_202609.md §1.2): only Views, scalars and KOKKOS_INLINE_FUNCTION helpers such as ww::jonswap (v, spectrum_fixtures.hpp): no std::vector, no std::function, no this, no allocation, no exceptions. One nvcc quirk the lab hit: an extended lambda cannot live inside a private member function, and a GoogleTest TEST() body is one, so kernels sit in free functions (v, kokkos/README.md).

12.8 Fences

Kokkos::fence() before the host reads a result, before a mirror dies, before a timer. 06_interop_bindc.cpp fences because the Fortran caller reads the array the instant the call returns (v); snl1_tables.cpp fences because its host mirrors go out of scope at the end of make_tables (v). Do not sprinkle them: each one destroys overlap.

12.9 float versus double

WW3's spectrum is default REAL; the port keeps float state on purpose, halving bytes per quadruplet (v, real.hpp). Every kernel literal is written static_cast<Real>(...) and the build turns on -Wconversion so a stray double is a warning, not a silent 2× in bandwidth (v). Accumulate in double. And know that x**n in gfortran is a chain of multiplications, not powf: snl1_tables.cpp has a powi() that reproduces the chain, and that is the difference between bit parity and "close" (v, kokkos/README.md).

12.10 Modern C++ hygiene, briefly

RAII: Kokkos::ScopeGuard guard(argc, argv) initialises and finalises the runtime with the scope (v, every intro main; 06 is a library and exposes init/finalize entry points instead). No raw new/delete, no C arrays in kernels, const by default, C++20 (v, kokkos/CMakeLists.txt). One warning policy for the whole tree, kokkos/cmake/CompilerWarnings.cmake applied to every target through ww_apply_warnings (v). The serial-debug preset adds AddressSanitizer and UndefinedBehaviorSanitizer (v, CMakePresets.json). Exceptions are allowed outside kernels only; the shim in lesson 12 turns them into status codes before Fortran sees them.

12.11 bind(C) interop

06_interop_bindc.cpp and 06_interop_driver.F90 are the smallest honest boundary (v). C++ side: extern "C" (no mangling, no overloads, no references), an unmanaged LayoutLeft HostSpace view over the caller's pointer (Kokkos neither allocates nor frees, Fortran stays the owner), and separate init/finalize entry points because Kokkos::initialize() may run once per process. Fortran side:

```
subroutine ww_intro_scale(n, x, s) bind(c, name='ww_intro_scale')
  import :: c_int, c_float
  integer(c_int), value :: n
  real(c_float), intent(inout) :: x(*)
  real(c_float), value :: s
end subroutine ww_intro_scale
```

VALUE for scalars C takes by value, iso_c_binding kinds everywhere, an assumed-size array so the address of the first element crosses and column-major order is preserved. The real shim (kokkos/src/fortran_iface/) is this with two-dimensional arrays, where the layout choice actually bites.

12.12 Building: presets and the CUDA backend

Preset	What it is (v, CMakePresets.json)
serial-debug	Debug, sanitizers, WW_DETERMINISTIC=ON
openmp-release	Release, -O3 -march=x86-64-v3
cuda-release	Release, CMAKE_CXX_COMPILER=nvcc_wrapper, CMAKE_CUDA_ARCHITECTURES=89

```
just kokkos-test openmp-release
just kokkos-cuda-test           # configure, build, ctest inside the #cuda shell
```

The pinned CUDA Kokkos is built for **one architecture only**, Kokkos_ARCH_ADA89: the RTX 4090. An H100 needs a Kokkos rebuild with HOPPER90 in nix-config, not a change to the presets (v, CMakePresets.json comment). Kokkos 5.2.0 is the pin (v, kokkos/README.md). Kokkos_ENABLE_DEBUG_BOUNDS_CHECK is an option of the Kokkos build itself, so the lab cannot switch it on from here; the sanitizers are the net (v).

Exercise: `exercises/solutions/ex12_reduce.cpp`, a `parallel_reduce` computing H_s from a spectrum View, built against `ww_kokkos`.

→ [12-porting-a-kernel-w3snl1.md](#)

Chapter 13

12. Porting a kernel: W3SNL1

Step 4 of the ladder, done once, end to end, on the routine lesson 07 nominated: the DIA nonlinear interactions. Everything here is in kokkos/; keep it open. The phase-1 rule is *translate, do not improve*: same expressions, same order, same float32 arithmetic. The only thing that changes is who executes them (v, kokkos/src/ww_kokkos/snl1_dia.

13.1 Source map

Source of truth: WW3/model/src/w3snl1md.F90 at 7.14 develop; line numbers (v, checked in ~/src/WW3 and in snl1_ref.F90's header). The C++ keeps the Fortran's numbered section comments:

Fortran	Lines	Sections
INSNL1	483–786; body 602–774	1 quadruplet angles · 2 lambda weights · 3
W3SNL1	115–473; locals 306–328; body 338–440	1 propagation constant · 2 auxiliary spectr

13.2 Config and Tables

ww::snl1::Config (v, snl1_config.hpp) is exactly the W3GDATMD inputs, under WW3's names: nk, nth, xfr, dth, lam, snlc1, kdcon, kdmn, snls1, snls2, snls3, fachfe, plus nspec(); SIG travels separately (host for INSNL1, device for W3SNL1). PI, TPI, TPIINV are built as constants. F90 builds them, PI rounded to REAL first (v). ww::snl1::Tables (v, snl1_tables.hpp) is INSNL1's output: nfr, nfrhgh, nfrchg, nspecx, nspecy, nspec, dal1..3, awg[8], swg[8], and 33 device Views: ip[2][4] and im[2][4] of length NSPECX, ic[8][2] of length NSPEC, af11 of length NSPECX. Arrays of Views because that is how W3SNL1 reads them, and because a View is a copyable handle, so the whole struct is captured by value into the kernel (v).

13.3 Index conventions

Fortran	Port
ISP = ITH + (IFR-1)*NTH, all 1-based	isp = ith + ifr*nth, all 0-based
IF3..IF6 clamped to 0 → addresses down to 1-NTH	entries as low as -NTH are legal; L1_test_sn1
UE(1-NTH:NSPECY), SA1(1-NTH:NSPECX) ...	scratch of length upper + NTH; Fortran index
A(NSPEC) one point per call	a(nspec, npts) LayoutLeft, column ip is one

13.4 Kernel structure

One **team per sea point** (TeamPolicy(npts, Kokkos::AUTO)), because the working set, an extended spectrum plus nine helper arrays, is per point and belongs in team scratch. The budget (NSPECY + NTH) + 8·(NSPECX + NTH) + NSPEC floats is checked against policy.scratch_size_max(0) before the launch; too big throws std::runtime_error (v). Inside the team (v):

1. **Section 1:** CONS from KDMEAN: three flops, recomputed per thread; cheaper than a barrier.
2. **Section 2:** TeamThreadRange over nfr fills ue and con; a second range zeroes slots [0, NTH) (the Fortran D0 ISP=1-NTH,0); team_barrier(). The tail D0 IFR=NFR+1,NFRHGH stays **sequential** in ifr, a barrier per row: row IFR reads row IFR-1.
3. **Section 3:** TeamThreadRange over nspecx: EP1, EM1, EP2, EM2, then SA1, SA2, DA1C..DA2M; team_barrier().
4. **Section 4:** TeamThreadRange over nspec writes s(isp, ipt) and d(isp, ipt).

No reduction anywhere: every output element is written by one thread from inputs no thread modifies, so WW_DETERMINISTIC changes nothing and the result is bit-identical across backends and thread counts; the launch is labelled "srce.sn11.dia" (v).

Two build facts decide whether "bit-identical" is true (v, kokkos/src/ww_kokkos/CMakeLists.txt, kokkos/README.md). **Floating-point contraction is off** for ww_kokkos: -ffp-contract=off, and -fmad=false for the device half under nvcc. GCC's default fuses AWG1*UE(..) + AWG2*UE(..) into an FMA (one rounding where the Fortran does two), and openmp-release drifted 1.1e-5 relative from the fixture while serial-debug was bit-identical; with contraction off all three presets reproduce the Fortran bit for bit, and removing that line is a physics change, not an optimisation. And **x**n is not std::pow**: powi() and pow11() reproduce gfortran's multiplication chains.

13.5 The shim and W3KOKKOSMD

kokkos/src/fortran_iface/ is the boundary: ww_kokkos_c.hpp declares the C ABI, sn11_shim.cpp implements it, w3kokkosmd.F90 mirrors it in ISO_C_BINDING blocks (v).

Entry point

```
int ww_kokkos_init(int comm_f)
void ww_kokkos_finalize(void)
int ww_sn11_init(nk, nth, xfr, dth, lam, snlcl, kdcon, kdmn, snls1, snls2, snls3, fachfe, s
```

Entry point

```
void ww_snll(npts, a, cg, kdmean, s, d)
int ww_snll_enabled(void)
int ww_snll_last_error(void)
```

Three things a naive extern "C" wrapper gets wrong and this one does not (v, snll_shim.cpp): ownership of the runtime (a test may own it); lifetime (a push_finalize_hook drops the Views inside finalize, never at process exit); errors (nothing throws across the boundary: every entry is a try/catch that records a code). W3KOKK05_SETUP turns ww_snll_enabled() into LOGICAL :: KOKK05_SNL1 once, so the inner loop reads a logical; every REAL is REAL(C_FLOAT) (v). L1_test_snll_shim.cpp drives the raw C API on the fixture with poisoned output buffers, so a forgotten copy-out fails loudly; shim_roundtrip (fixtures/shim_driver.F90) is the same call from Fortran (v, PORT_STATUS.md).

13.6 Fixtures

The L1 tests do not compare against numbers a human typed (v, kokkos/tests/fixtures/README.md)

File	What
snll_ref.F90	verbatim W3SNL1/INSNL1 bodies; the only edits are the USE lines replaced
gen_snll_fixture.F90	SETUP_REF(25, 24, 1.1, 0.04118) with ST4/NL1 defaults, a JONSWAP (1
snll_nk25_nth24.bin	the committed result, 107 892 bytes, little-endian, no record markers, t

The three depths exercise the KDMEAN branch: deep water where EXP(X2) underflows, and a 7 % correction at 10 m (v). just snll-fixtures regenerates the file, and a byte-level diff then needs a reason in the commit message (v). fixture_io.{hpp,cpp} is the one reader (v).

13.7 L1 tests and their tolerances

Suite	Checks	Tolerance and why
L1_test_snll_tables	all 32 address tables dal1..3, awg, swg, af11	exact: an address is an integer 1e-6 relative: they come out of a table
L1_test_snll_dia	S and D on three points zero spectrum → zero source; A×2 → S×8, D×4 four relaunches	1e-5 relative with a 1e-30 absolute properties of the DIA, not of the source bit-identical: a difference in the source

Why 1e-5 for float32: the state has 24 bits, about 6e-8 relative, and section 3 forms products of four-term interpolations scaled by AF11 over ten decades, so a handful of ULPs is the honest floor. 1e-5 is two orders above that and *tight enough that one fused multiply-add fails it* (the FMA drift measured 1.1e-5). Measured: 0.0 on all three presets (v, PORT_STATUS.md).

13.8 The optional ww3_lib cross-check

gen_snll_ww3lib.F90 and just ll-crosscheck would prove snll_ref.F90 verbatim by running the *real* W3SNL1 from a configured WW3 build and cmp-ing the fixtures. **Neither has been run:** the owner's WW3 build used NL0; it needs an NL1 build such as just build switches/switch_lab_shrd (v, kokkos/README.md).

13.9 PATCH.md: what changes in the WW3 caller

w3kokkosmd.F90 is written to be dropped into WW3/model/src unchanged (v, its header). The caller side is kokkos/src/fortran_iface/PATCH.md: a hunk-by-hunk recipe for a fork branch, with real line numbers, not applied in this repo (v). Four hunks (v):

1. **w3srcemd.F90:** inside the W3_KOKKOS guard, USE W3KOKKOSMD, USE W3SERVMD, ONLY: EXTCDE and USE W30DATMD, ONLY: NDSE (neither is otherwise in scope in W3SRCE); a one-element KDM_K(1) local, because KDMEAN there is the expression WNMEAN*DEPTH; and at the single call site (lines 1258-1264) IF (KOKKOS_SNL1) THEN CALL WW_SNL1(1, SPEC, CG1, KDM_K, VSNL, VDNL) ... ELSE CALL W3SNL1(...) END IF. The call sits in a named !\$OMP CRITICAL: W3SRCE runs inside an !\$OMP PARALLEL region and the shim has one unlocked context, so phase 1 serialises it, slower than the Fortran when threaded; the correctness step, not the fast one. The WW_SNL1_LAST_ERROR() /= 0 check followed by EXTCDE is **not optional**: a void C call has no other channel, and a silent failure is a plausible-looking wrong forecast. CG1 is already CG(1:NK, ISEA), sliced by W3WAVE, so it passes straight through.
2. **w3initmd.F90,** after W3IOGR('READ'): WW_KOKKOS_INIT(-1), W3KOKKOS_SETUP, EXTCDE when IMOD > 1 (one spectral grid per process: ww_snll_init *replaces* the tables), then WW_SNL1_INIT(..., SIG(1)). SIG(1), not SIG: W3GDATMD allocates SIG(0:MK+1) and the assumed-size C dummy would start at bin 0: every quadruplet one bin low, smooth, plausible and wrong; the lab's SIG(NK) reference cannot catch it.
3. **switches.json** gains a KOKKOS category with "build_files": ["w3kokkosmd.F90"] and "requires": ["NL1"]. src_list.cmake needs **no** change: listing the file there would compile it unconditionally and break every non-Kokkos build.
4. **model/src/CMakeLists.txt** links ww_kokkos under -DWW_KOKKOS=ON and refuses the option without the switch.

Why the fork carries it: WW3's physics is frozen except for shims (AGENTS_KOKKOS §5), and the lab pins WW3 as a submodule fork (just src-init, src-pr, src-sync (v, justfile)), so the patch lives on a branch the pin can point at, and upstream stays untouched.

13.10 L2: replay a regtest through both paths

```
just l2 ww3_ts1          # kokkos/tests/L2_replay.sh <ww3-dir> <regtest>
```

Requires just rt <regtest> done. The script copies work_lab to work_a and work_b, runs ww3_shel with WW_KOKKOS_SNL1=0 and =1, runs ww3_ounf in each, then nccmp-tol

work_a/ww3.nc work_b/ww3.nc with the default tolerances, and appends the table to kokkos/PORT_STATUS.md under a ## L2 replays heading it creates (the ledger's own section is ## L1 / L2). Until the patch is applied on a fork branch both runs are the Fortran path and the table shows zeros: that proves the harness, not the kernel, and the ledger's L2 column reads "pending fork branch" (v).

13.11 The timing line

kokkos/PORT_STATUS.md is the ledger, one row per routine: WW3 file and lines, phase, shim, L1 parity, L2 replay, Serial / OpenMP / CUDA ms per call, notes (v). The W3SNL1 row comes from ww_bench_sn11 (kokkos/tests/bench_sn11.cpp, not a test: 1 000 sea points, 20 timed calls after 3 warm-ups, median of three runs), on an Intel i9-14900 + RTX 4090, Kokkos 5.2.0, GCC 15.3 (the owner's workstation, not the CI runner) (v):

backend	through the shim, ms/call	kernel only, ms/call	vs Serial
Serial (OpenMP backend, 1 thread)	25.88	24.96	1.0x
OpenMP, 32 threads	5.95	5.02	4.4x
CUDA, RTX 4090	0.75	0.047	34x

Two things these numbers are not (v). Not the fastest the kernel can go: every row is the -ffp-contract=off parity build, and turning contraction on would invalidate the L1 column. And not what WW3 would see today: the phase-1 call site passes NPTS = 1, and on CUDA 94 % of the 0.75 ms call is host→device copy: the entire argument for phase 2, and why "34x" is not the model's speed-up.

13.12 What "done" means

docs/AGENTS_KOKKOS_202609.md §1.5, which the course spec adopts: (1) the kernel with a heritage header naming the WW3 routine, (2) the bind(C) shim and Fortran interface with the argument table, (3) an L1 test with tolerances stated and justified, (4) an L2 replay of the smallest regtest that exercises it, (5) a timing line in PORT_STATUS.md, (6) a property test where physics allows: for Sn1, cubic scaling and zero-in/zero-out stand in until an action-conservation test exists $\triangle$. Items 1-3 and 5 are in the tree (v); 4 waits on the fork branch. Sheet: exercises/ex13_port.md, solution exercises/solutions/ex13_compare.sh.

→ [13-bulk-porting-with-agents.md](#)

Chapter 14

13. Bulk porting with agents

Lesson 12 ported one routine by hand, with a coding agent as the typist. This lesson is about doing it forty more times without losing what made the first one trustworthy: every step compared against captured Fortran, every claim with a table attached. The rules come from docs/AGENTS_KOKKOS_202609.md (written to be dropped in as AGENTS.md), the plan from the proposal, and the diagrams from its mind maps (pubs/proposal/mapas-mentais.pt.md).

14.1 The FESOM2 recipe

Koldunov et al. (2026) took FESOM2 (about 74 thousand lines of Fortran) to C and then to C++/Kokkos in **weeks, not years**, with an LLM coding assistant directed by the model's own experts (pubs/proposal/pt/05-justification.md (v); details beyond that summary $\triangle$). The recipe:

1. **Two stages.** Fortran $\rightarrow$ a clean, single-threaded C reference $\rightarrow$ C++/Kokkos. The C stage pins the configuration (dead `#ifdefs` disappear) and is what the Kokkos serial backend is compared against.
2. **Literal translation.** Same expressions, order and precision; optimisation is a later, separate change.
3. **A validation ladder.** Each rung compares against the rung below on captured inputs.

flowchart LR

```
A[Original Fortran] -- literal translation,<br/>LLM assistant directed<br/>by the exper
B -- checked against the Fortran<br/>on captured inputs --> B
B -- expressed in Kokkos --> C[Kokkos, serial backend]
C -- must be bit-identical<br/>to the C --> C
C -- same code --> D[Kokkos, CUDA backend]
D -- statistical comparison<br/>and timing on the H100 --> D
subgraph T[Tests per kernel]
    L1[L1 · synthetic JONSWAP spectrum<br/>declared tolerances]
    L2[L2 · nearest regression case<br/>and the operational case]
end
C --> T
D --> T
```

The lab collapsed the C stage for W3SNL1 (100 lines, already CPP-preprocessed .F90): the "bit-identical to the C" rung became "bit-identical to the Fortran fixture on every preset" (lesson 12). For W3SRCE and its 239 preprocessor guards Δ , keep the C stage.

14.2 What a port PR contains

The six items of lesson 12's definition of done (AGENTS_KOKKOS §1.5, §5): kernel with heritage header; bind(C) shim plus Fortran interface; L1 test with justified tolerances; L2 replay of the smallest regtest; a PORT_STATUS.md timing line; a property test where physics allows. Any algorithmic change is a separate PR with its own L2 evidence (v).

14.3 The ranked list and the phases

AGENTS_KOKKOS §2.2 scores each routine by runtime share $\times$ Kokkos suitability $\times$ ensemble payoff $\div$ (effort + validation risk), with the shares as priors until §2.4's profiling (lesson 09) re-sorts it (v). Summarised:

#	Routine(s)	Kokkos shape
1	W3SNL1 + INSNL1	team per point, tables shared
2	W3SRCE driver	one team per point calling in
3	W3SIN4, W3SDS4, W3SPR4 (or ST6)	inline device functions inside
4–5	W3XYP2 + W3QCK*; W3KTP2/3	MDRangePolicy<Rank<3>> per m
6	W3GATH/W3SCAT	eliminate: one decomposition
7	W3SBT*, W3SDB1, W3STR1, W3SIC*	trivial once #2 exists
8–9	output integrals (w3iogomd); forcing interpolation (w3updtmd)	parallel_reduce on device, c
10	restart and field I/O	not Kokkos work: async, gro

PDLIB and the exact/GMD Snl variants are explicitly last (v). Phases (§3.6): 0 profile and ledger; 1 W3SNL1, ST4, W3SRCE skeleton with copy-in/out, exit on L2 parity on three regtests; 2 device-resident state; 3 propagation and halos on device; 4 the member dimension; 5 flip to a C++ driver (v).

14.4 Interoperability, one binary

```

flowchart TB
    subgraph BIN[One WW3 binary]
        direction LR
        F[Original Fortran routine]
        SW{Runtime switch}
        C[bind C interface<br/>ISO_C_BINDING]
        K[Kokkos kernel<br/>Views in the layout of<br/>the WW3 spectral arrays]
        SW -- original path --> F
    end

```

```

    SW -- new path --> C --> K
end
K --> CPU[Serial or OpenMP backend<br/>no data copy]
K --> GPU[CUDA backend on the H100<br/>CPU-GPU traffic measured per step]
BIN --> M[WW3 regression matrix<br/>and per-field comparator<br/>run both paths without

```

This is W3K0KK0SMD and WW_K0KK0S_SNL1 from lesson 12, drawn; “measured per step” is ww_bench_snll’s shim-versus-kernel gap.

14.5 The data-residency ladder

AGENTS_K0KK0S §3.4 (v), the plan for making the phase-1 copy disappear:

Step	On the device	Crosses per time step
1	nothing persistent; VA copied in and out per call	full spectra, twice
2	VA, grid metadata, tables; source terms on device	forcing (small), output fields (small)
3	+ propagation with device halo buffers	halos, forcing, output
4	+ member dimension	same, batched

Step 2 is where the OpenACC port of Ikuyajolu et al. (2023) stalled, because the state lived in Fortran modules; WAM6-GPU (Yuan et al. 2024) went the other way, refactoring the whole model so the fields stay on the device, and reported an order-of-magnitude gain on a multi-GPU node (pubs/proposal/pt/05-justification.md (v)). The plan does step 2 in C++ ownership (a state object created by ww_kokkos_init, seeded by the Ctx in snll_shim.cpp with its persistent buffers) rather than mirroring the modules (v).

14.6 The ladder and its gates

```

flowchart TD
    R[Frozen reference run<br/>code, switches, namelists, grid, forcing] --> B[Reproducible]
    B --> P[Profile by routine and by phase<br/>1, 4 and 16 MPI processes]
    P --> E1
    subgraph E1[Step 1 · Compile options]
        direction LR
        E1a[compiler, flags, switches,<br/>MPI x OpenMP] --> E1g{bit for bit<br/>or rounding}
    end
    E1g -- yes --> E2
    E1g -- no --> X1[discarded]
    subgraph E2[Step 2 · Run configuration]
        direction LR
        E2a[decomposition, time steps,<br/>outputs, restart, forcing] --> E2g{WW3 matrix<br/>or forcing}
    end
    E2g -- yes --> E3
    E2g -- no --> X2[discarded]

```

```

subgraph E3[Step 3 · Modern Fortran]
  direction LR
  E3a[routines at the top of the profile,<br/>one at a time, same arithmetic] --> E3g
end
E3g -- yes --> E4
E3g -- no --> X3[discarded]
subgraph E4[Step 4 · C++/Kokkos kernels]
  direction LR
  E4a[only routines still dominant<br/>after step 3] --> E4g{measured gain<br/>and pa
end
E4g -- yes --> OP[Enters the operational configuration]
E4g -- no --> LIM[Measure of the limit,<br/>recommendation not to operate on GPU]

```

Lessons 09, 10 and 12 are rungs; the gates are the matrix, nccmp-tol and the fixture tests. An agent may work on any rung; it may not skip one.

14.7 The operation decision

```

flowchart LR
  K[Kernel rewritten<br/>in C++/Kokkos] --> S1{Kokkos serial<br/>bit-identical to the C?}
  S1 -- no --> F1[fix the translation]
  S1 -- yes --> S2{Parity with the Fortran<br/>on the operational case?}
  S2 -- no --> F2[not incorporated]
  S2 -- yes --> S3{Operational case faster<br/>with the kernel on the H100?}
  S3 -- no --> F3[Report of the limit:<br/>CPU-GPU traffic per step]
  S3 -- yes --> D[Decision with LabEC0:<br/>timing table + parity report]
  D --> OP[Operational configuration]

```

Two documents, one meeting: the timing table of the operational case with and without the kernel, and the parity report. If the H100 does not win, the deliverable is the measured limit and a recommendation *not* to operate on the GPU (pubs/proposal/pt/07-methodology.md (v)).

14.8 Prompting an agent: patterns and anti-patterns

Give it, every time (AGENTS_KOKKOS §1.6 (v)): the exact routine with file path and WW3 commit; the regtest that exercises it and the reference output; the interface contract (which arrays cross, in which layout, which precision); and the phase, so it knows which optimisations are off limits. The document's own example is the prompt behind lesson 12: "Port W3SNL1 (DIA) from model/src/w3snl1md.F90 @ WW3 7.14 ... Phase 1: translate only ... Write L1 test against a JONSWAP fixture ... Do not touch the Fortran caller except to add the bind(C) interface behind the WW_KOKKOS_SNL1 switch."

Forbid explicitly (v): optimising while translating; replacing preprocessor switches with runtime ifs inside kernels; claiming a speed-up without an attached timing table; editing WW3 physics to "make interop easier"; introducing Unified Memory

to avoid thinking about transfers. And require it to *ask* before changing a loop order that affects reduction order, a precision, a limiter or an integration scheme (§5).

Two lab-specific lessons from the first port (kokkos/README.md (v)): an agent reaches for `std::pow` where gfortran multiplies, and for a fused multiply-add where the Fortran rounds twice. Neither is wrong C++; both fail a 1e-5 parity test. Build the fixture first.

14.9 WW4: do not compete, build in its shape

WAVEWATCH IV is NOAA's rewrite (plan: Office Note 525, 2025; Phase II, languages, governance, architecture, closed March 2026, Office Note 528). As of 2026-09-15 it has a C++ core with no physics, L1 and L2 tests in GoogleTest of four planned levels, and an open CPU-GPU architecture question with Kokkos proposed as the abstraction layer (pubs/proposal/pt/05-justification.md (v); [lesson 14](#)). ON 525 retires WW3 only once WW4 matures, so WW3 stays operational for years. The project's artefacts are shaped to be reusable there (L1 per-kernel tests on synthetic spectra, L2 replays, GoogleTest, heritage headers) without contributing to WW4 or depending on it (pubs/proposal/pt/04-scope.md (v)). Licence (AGENTS_KOKKOS §4): the translated kernels derive from WW3, so `snl1_dia.cpp`, `snl1_tables.cpp` and the Fortran module that patches into the model, `w3kokkosmd.F90`, carry LGPL-3.0-or-later; the C++ tooling around them, `snl1_shim.cpp` and the C header included, is MIT (v); and nothing here is called WW4.

14.10 Sources

- Koldunov et al. (2026), FESOM2 Fortran → C → C++/Kokkos: <https://doi.org/10.48550/arXiv.2606.11356>
- Ikuyajolu et al. (2023), *GMD* 16, 1445–1458: <https://doi.org/10.5194/gmd-16-1445-2023>
- Yuan et al. (2024), WAM6-GPU v1.0, *GMD* 17, 6123–6136: <https://doi.org/10.5194/gmd-17-6123-2024>
- NCEP Office Note 525 (2025): <https://doi.org/10.25923/h7j3-1h25>; Office Note 528, Tolman (2026): <https://doi.org/10.25923/0wyp-9f39>; keys in `pubs/proposal/refs.bib`

→ [14-ww4-and-the-future.md](#), or [15-swan.md](#).

Chapter 15

14. WW4: what's coming, and how mature it is

Short version: WW4 is real, it is a full ground-up rewrite in C++ (with Rust in parallel) rather than a new WW3 version, and as of September 2026 it is a driver skeleton with a test framework and no physics. Learn WW3. Watch WW4. Build your artefacts in the shape WW4's tests already have.

15.1 It exists

- Repository: [NOAA-EMC/WW4](#) (v): "Home of the WAVEWATCH IV™ (WW4™) third-generation wind wave modeling framework", created 2025-11-17 (v).
- Planning documents: **NCEP Office Note 525**, *The WAVEWATCH III® Software Modernization Project: Phase I report*, Hendrik L. Tolman, November 2025, [doi:10.25923/h7j3-1h25](#) (v); and **NCEP Office Note 528**, *The WAVEWATCH IV Project: Phase II report*, Tolman, 2026, [doi:10.25923/0wyp-9f39](#) (v).

Office Note 525 is the single best thing to read if you want to understand where wave modelling is going. It is unusually candid: it publishes the disagreements inside the discussion group rather than papering over them. ON 528 closes Phase II.

15.2 Maturity: a snapshot

Checked against the repository on **2026-09-15** (v):

Signal	Value
Default branch	develop
Commits	37
Releases / tags	none ; VERSION reads 0.0.0
Stars / forks / watchers	3 / 7 / 4
Open issues / open PRs	31 / 0

Signal	Value
Last push	2026-09-15: the merge of PR #66, "CMake-only compile system", open
Top-level	src/ tests/ tools/ templates/ externals/, plus AGENTS.md, ARCHITECTURE
src/	ww4_core/ (w4core_init, w4core_wave, w4core_finalize), ww4_utils/, ww4_
tests/	GoogleTest; four declared levels, L1 and L2 present, L3 and L4 absent

Thirty-seven commits, ten months in, and no release. This is scaffolding and early core work, an init/wave/finalize driver with a standalone program around it, not a model you can run. Treat it as a project to follow, not a tool to adopt.

What has happened this year, in order (v): PR #50 (2026-07-22) brought the unit (L1) and integration (L2) tests to full coverage of the code that exists; issue #43 (2026-07-15, still open, no comments) proposes the **CPU-GPU architecture**, naming Kokkos as the data abstraction and MPI between nodes, and asks whether to keep one code base for CPU and GPU; PR #60 (2026-08-19) removed the Python dependence from the build; PR #66 made the build CMake-only. Around thirty open issues are design questions of that kind, not bugs.

△ Those numbers were true on one day, literally: the snapshot this lesson was drafted from, earlier the same day, still showed 36 commits, 35 open issues and PR #66 open. Check them yourself before quoting them; that is exactly the kind of thing that goes stale fastest.

15.3 Why a new model rather than WW3 v8

The Phase I report lays out the drivers plainly:

- **Parallel concepts are 24 years old.** WW3 uses a "shuffle" method for distributed computing (Tolman 2002a) that load-balances dynamic source-term timestepping beautifully but doesn't scale to exascale. WW4 moves to conventional domain decomposition.
- **Data structures are rooted in the Fortran 90 transition** of nearly two decades ago. WW4 wants spatial data structures local to each domain, for scalability and a smaller memory footprint, and for source-term integration over *areas* rather than individual grid points.
- **Optimisation now means memory, not FLOPs.** The report says this directly: the rise of GPUs and other advanced architectures requires focusing on memory use and access, complementing the traditional focus on floating-point operations. (Lessons 11–13 are the hands-on version of the same conclusion: where the spectrum *lives* decides the speed-up.)
- **Fractional stepping fights implicit schemes.** WW3's architecture leans on fractional steps (Yanenko 1971), which works beautifully for explicit propagation and badly for the implicit unstructured-grid solvers everyone now wants.
- **Closer coupling.** UFS wants component models callable at the level of functional units, not wrapped as monoliths. That needs an object-oriented interior.
- **The Fortran compiler pool is shrinking.** Stated as a continuity-of-operations risk: NWS wants to move off Fortran *in a controlled way, before it becomes urgent*.

- **Regression testing has become unsustainable.** Recompiling between individual tests is called out as untenable. WW4 moves to unit and integration testing.

The decision was a **complete bottom-up rewrite in a new repository**, explicitly modelled on how NOAA handled MOM6 (developed separately from MOM4 rather than as an increment). A new repo removes the obligation of backward compatibility and allows “house cleaning” of options that no longer have an owner.

15.4 Languages: the contentious part

The report says outright that language choice was the most contentious question, and that **there is no community consensus**. NOAA/NWS made the call as primary funder:

Language	Role in WW4
C++ (likely with Kokkos)	Initial core language. Conservative, proven, operational centres already using it.
Rust	Named “the modern language of choice for WW4” by NOAA/NWS, but not chosen.
Fortran	Solver language only , plus a fast route to an initial operational capability.
Python	Scripting, workflow, data management, product generation, grid generation.
Julia	Considered and declined as a core language by NOAA/NWS: small community, no proven track record.

The chosen path is the “**dual approach**”: build a C++ core to an operations-ready state in roughly two years, while incrementally building Rust alternatives; a Rust-cored WW4 for operations is a roughly five-year target. What the repository shows in September 2026 is consistent with that: the core is C++, and the open architecture issue proposes Kokkos. The “likely” in the table above has become a written proposal, not yet a decision (v).

A design principle worth noting, because it's the thing that would let you contribute: WW4 is deliberately keeping a **clean separation between core and solver code**, with language bindings between them, so a contributor writing a new source-term parameterisation can do it in whichever language they're fluent in without learning the core language.

15.5 Yes, it is being written partly by AI agents

The WW4 repository contains an `AGENTS.md` describing “an agentic AI approach used to create, translate or refactor code using AI agents such as Copilot or Jules, the latter of which has been used extensively in developing the WW4 code from WW3.” (v) The agent is also set up to enforce coding standards, generate doxygen documentation, and write unit tests, all of which are mandated for WW4.

Worth knowing, for two reasons. It's a large, visible, government-operational test of AI-assisted translation of scientific Fortran. And it means the provenance of any given line in WW4 is a live question you should keep in mind when reading it. Lesson 13 is this repo's own, much smaller, version of the same experiment, with the parity evidence versioned next to the code.

15.6 Timeline

From Office Note 525 (v), with the report's own caveat that timelines for a project like this are notoriously difficult, and a fourth column for what the repository actually shows:

Phase	What	When (ON)
I	Initial choices	complete (C
II	Language test, governance setup, architecture design	began 1 Oc
III	Core code development (EMC)	open sourc
IV	Initial Model Capability: single domain, CPU <i>and GPU</i> efficiency focus	active comm
V	Initial Operational Capability, multi-domain	
VI	Complete first full code	first public

Those dates are for the C++ path. Rust may take up to five years.

15.7 What WW4's test levels mean for a WW3 port

WW4's tests/README.md declares a four-level strategy (v): **L1** unit tests of single functions in isolation (`L1_test_*`), **L2** integration tests of modules and their interfaces (`L2_test_*`), **L3** functional tests of physics cases as a black box, and **L4** regression tests of the full model. L1 and L2 exist, in GoogleTest, at full coverage of the code that exists; L3 and L4 do not yet, because there is no physics to test.

This repo copies that structure on purpose, and the proposal says why: *do not compete with WW4; build artefacts in the shape WW4 can use*. So the W3SNL1 port in [12-porting-a-kernel-w3snl1.md](#) comes with `L1_test_*` GoogleTest suites that compare the Kokkos kernel against captured Fortran fixtures, and an `L2_replay.sh` that reruns a whole WW3 regtest with the kernel on and off and judges the netCDF output with `nccmp-tol`. Same prefixes, same framework, same meaning of the levels. When WW4 reaches the point of needing a DIA kernel and a way to prove it matches WW3, the fixtures, the tolerances file and the parity report are the deliverables it can pick up, and if it never does, they are still the evidence the lab needs to put the kernel into operation. Either way the work is not wasted, which is the whole point of choosing the shape before choosing the kernel.

15.8 What this means for WW3: and for you

WW3 is not going away soon, but its end is now scheduled. The report is explicit: the cost of maintaining two models long-term will be mitigated by “formally sunseting most support for WW3 once WW4 is mature, with a clearly communicated transition period.” Code with no identified owner willing to port it stays in WW3 and is considered obsolete for WW4.

Practically:

- **Learn WW3 now.** It is the mature, documented, validated, operational model, and it will be for years. Everything in this repo remains the right thing to learn.

- **The concepts transfer completely.** The action balance equation, source-term packages, spectral discretisation, CFL limits, grids, nesting, partitioning: none of that changes. WW4 is a software rewrite, not new physics.
- **The interfaces will not transfer.** Expect namelists to go (the report canvasses ASCII / YAML / namelist and notes the choice follows from the language), expect the binary `mod_def.wv3` / `out_grd.wv3` files to go (consensus to move to NetCDF, with interest in Zarr), and expect the compile-time switch file to be reconsidered: the pushback from researchers about recompiling between runs is recorded in the report.
- **The separate-executables workflow is under review.** `ww3_grid` / `ww3_prep` / `ww3_shel` / `ww3_ounf` as distinct programs is explicitly listed as a design decision to revisit.
- **If you were going to GPU-port WW3 with directives: don't.** WW4 Phase IV targets CPU *and* GPU efficiency in a code architected for it from the start, and any heroic OpenACC work on WW3 has a short shelf life. What *does* keep its value is a kernel written in the abstraction WW4 itself is converging on, validated in WW4's own test shape, which is what lessons 11–13 do, and why they use Kokkos rather than `!$acc`.

15.9 One thing that must survive

The report singles out a property of WW3 that WW4 has to keep: **full numerical convergence of its schemes**, achieved through the limiter formulation of Tolman (2002b). Without it you cannot separate numerical error from physical error, which makes the model useless for science. It's a good reminder that "modernisation" has constraints that aren't about software at all.

→ [15-swan.md](#): the other model you should know.

Chapter 16

15. SWAN: the other one you should know

If you only ever learn WW3 you will eventually reach a coastal problem where WW3 is the wrong tool and you won't recognise it. SWAN is that tool. It's free, it's a few hours to build, and knowing when to reach for it is a real skill.

16.1 What it is

SWAN (Simulating WAVes Nearshore) is a third-generation spectral wave model from **Delft University of Technology** (Booij, Ris & Holthuijsen). Same governing equation as WW3, same family of source terms, deliberately different numerics.

Current version: **Cycle III version 41.51** (v). Free and open source.

16.2 Where the source actually lives

Three places, and the third is new enough that most tutorials don't mention it:

Source	URL
Official site + downloads	https://swanmodel.sourceforge.io/download/download.htm
SourceForge file releases	https://sourceforge.net/projects/swanmodel/files/swanmodel/
Git, TU Delft GitLab	https://gitlab.tudelft.nl/citg/wavemodels/swan (v)
Release notes / modifications	https://swanmodel.sourceforge.io/modifications/modifications.htm
Implementation manual (build guide)	https://swanmodel.sourceforge.io/download/zip/swanimp.htm

```
git clone https://gitlab.tudelft.nl/citg/wavemodels/swan.git && cd swan
```

Its sibling **SWASH** (non-hydrostatic, phase-resolving, same group) is at <https://swash.sourceforge.io/> and <https://gitlab.tudelft.nl/citg/wavemodels/swash> (v).

Ignore the various GitHub mirrors and vendored copies you'll find. They're snapshots of whatever version someone needed years ago.

16.3 Building it

Modern SWAN builds with **CMake 3.12+**, and the implementation manual recommends **Ninja** as the generator because it's faster than GNU make (v).

```
git clone https://gitlab.tudelft.nl/citg/wavemodels/swan.git && cd swan
cmake -G Ninja -B build -S .
cmake --build build
```

The older workflow (make config then make ser / make omp / make mpi) still exists and is what most older tutorials assume. Either works; CMake is where it's going.

scripts/04_get_swan.sh in this repo does the clone and build.

16.3.1 Switches, SWAN-style

SWAN has the same *idea* as WW3's switch file (compile-time feature selection) but a different mechanism. Options live as **specially-formatted comments inside the .ftn sources**, stripped or activated by a preprocessing step. For example `!/impi` marks MPI code in `swmod1.ftn`, and `!ADC` marks the ADCIRC-coupling hooks scattered across several files (v).

Practical consequence, identical to WW3: if you change what's enabled, you re-build. And the same class of confusion applies: "I turned on MPI and nothing happened" is a stale build directory.

16.4 SWAN versus WW3: the actual difference

Both solve the action balance equation. Both have DIA quadruplets, whitecapping, bottom friction, depth-induced breaking. The difference is **numerics**, and it decides everything else.

	WW3	SWAN
Propagation scheme	Explicit (PR3 UQ, third-order ULTIMATE QUICKEST)	Implicit sweeps
Timestep	CFL-limited. Halve Δx , halve Δt .	No CFL constraint
Sweet spot	Global and basin scale, deep water, long integrations	Coastal, high res
Stationary mode	no	yes: solve directly
Triads (3-wave)	TR0/TR1, limited	Mature; matters
Diffraction	no	approximate (ph)
Obstacles	subgrid obstruction grids	explicit obstacle
Ice physics	IC1-IC5, IS1/IS2, far ahead	minimal
Multi-grid mosaic	ww3_multi, two-way	no
Parallelism	MPI "shuffle" decomposition, OpenMP	OpenMP and MPI

The one sentence that matters: WW3's explicit scheme means the timestep shrinks with the grid spacing, so a 50 m coastal grid becomes ruinously expensive; SWAN's implicit scheme doesn't care. That is why the standard architecture in

coastal work is **WW3 offshore** → **SWAN nearshore**, with WW3 spectra as SWAN's boundary condition.

Note the trade: SWAN buys unconditional stability at the cost of numerical diffusion and, in non-stationary mode, accuracy that depends on how many sweeps you let it do. The implementation manual's advice on block-Jacobi versus block-wavefront is precisely about where that trade lands on a parallel machine. WW3's convergence property (the thing Office Note 525 insists WW4 must preserve) is not free, and SWAN made the other choice.

16.5 When to use which

Situation	Reach for
Global or basin-scale hindcast/forecast	WW3
Operational forecasting inside UFS/NOAA	WW3
Sea ice, marginal ice zone	WW3
Bay, estuary, harbour, surf zone at 10–200 m	SWAN
Stationary design wave conditions for engineering	SWAN (stationary mode)
Nearshore circulation via radiation stress	SWAN, or SWAN inside Delft3D
Wave-by-wave runup, overtopping, infragravity	SWASH or XBeach, not either of these
Coupled surge + waves, US practice	ADCIRC+SWAN
Coupled ocean–atmosphere–wave–sediment	COAWST (has both)

16.6 The ecosystem around it

- **Delft3D-WAVE** wraps SWAN and couples it to Delft3D-FLOW. Open source.
- **ADCIRC+SWAN** is the US storm-surge standard; the coupling hooks are in SWAN's source behind the !ADC switch.
- **rompy-swan** (v): pydantic-validated, type-safe SWAN configuration from Python or YAML, with data interfaces for NetCDF and THREDDS inputs. This is the most polished Python front-end for *any* spectral wave model, and noticeably more mature than anything equivalent for WW3.
- **wavespectra** (v) reads SWAN spectra natively (read_swan), so your post-processing is shared between the two models.
- **swantools** (PyPI): older, lighter; reads TABLE, SPECOUT and BLOCK output into pandas.
- **OMUSE** packages SWAN for the Oceanographic Multi-purpose Software Environment.

16.7 Exercise

Take examples/02-regional-real-forcing and extend it into a two-model chain:

1. Run WW3 on the 0.1° shelf grid as you already do.
2. Use ww3_ounp to write 2D spectra along a line just offshore of Florianópolis.

3. Build SWAN, set up a ~ 50 m nested grid over the bay, and feed it those spectra as a boundary condition.
4. Compare nearshore H_s against running WW3 alone at the same 50 m resolution.

You will learn three things: that the WW3-only run takes far longer for the same domain (the CFL limit, made personal); that they don't agree; and that converting spectral boundary data between two models' conventions is where the real work in coastal modelling actually goes.

→ Back to [README.md](#).